

Seq2Seq Models and Machine Translation

Rasha Obeidat, PhD

NLP

Rasha Obeidat

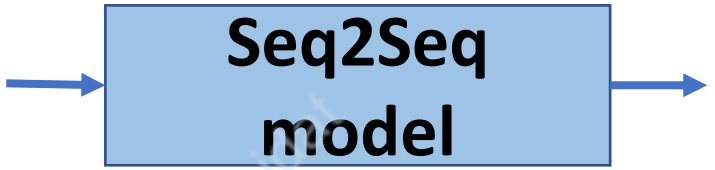
Seq2Seq models

- **Sequence-to-sequence model (Seq2Seq)** is including ***encoder-decoder*** based neural architecture that converts sequences from one domain (e.g. sentences in English) to sequences in another domain (e.g. the sentences translated to Arabic).
- Seq2Seq has been successfully applied to several learning tasks:
- Text Summarization: long text → Short text
- Machine Translation: Sentence → Sentence
- Image Captioning: Image → Textual description
- Code generation (natural language → Python code)
- Chatbots: Previous utterances → Next utterance

Application of Seq2Seq model

- **Machine Translation**

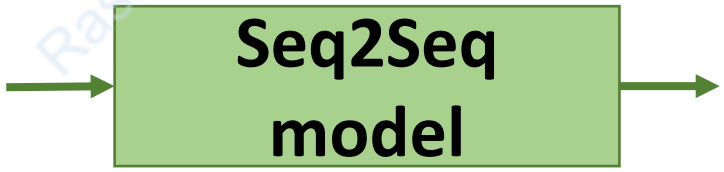
My youngest son loves playing football



يحب ابني الصغير لعب كرة القدم

- **Text Summarization**

أحمد شوقي شاعرٌ مصريّ من أعظم وأشهر الشّعراء العرب في جميع العُصور ولذلك لقّب أحمد شوقي بأمير الشّعراء. لقد تميّز الشاعر أحمد شوقي عن غيره من الشعراء بأسلوبهُ الشّعريّ الذي كان قريباً من الواقع والأحداث السّياسيّة في الفترة التي كان يعيشُ فيها، ولم يكُن فقط شاعرٌ سياسيّ فقط بل كان شاعرُ غزل يستمدُّ أفكارهُ من الخيال والعبارات الدافئة،.....



أحمد شوقي شاعرٌ سياسيّ و شاعرُ غزل تميّز شّعره بالواقعيّة ..

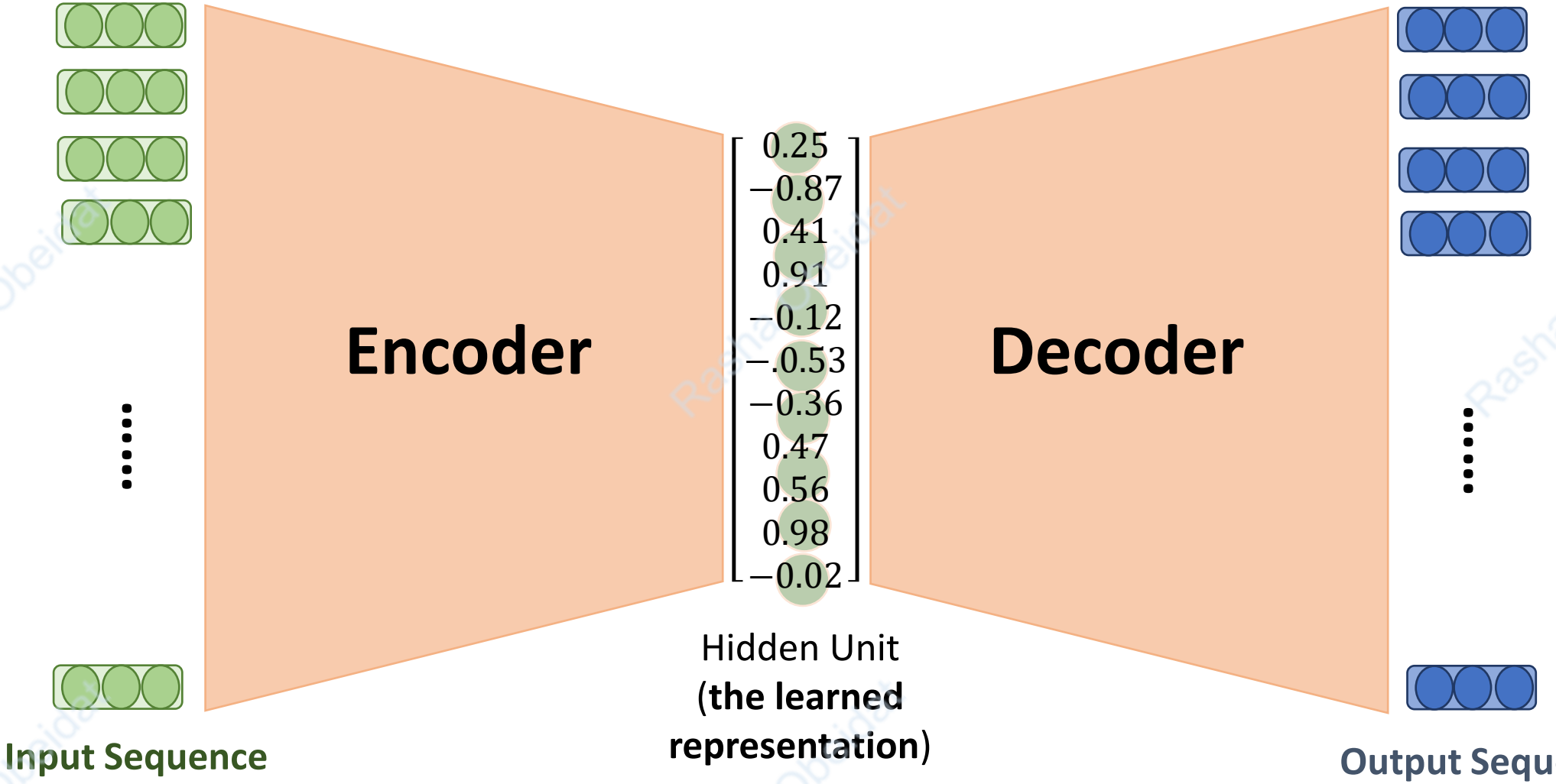
- **Chatbots**

هل يمكنك ان تساعدني في معرفة رصيدي؟



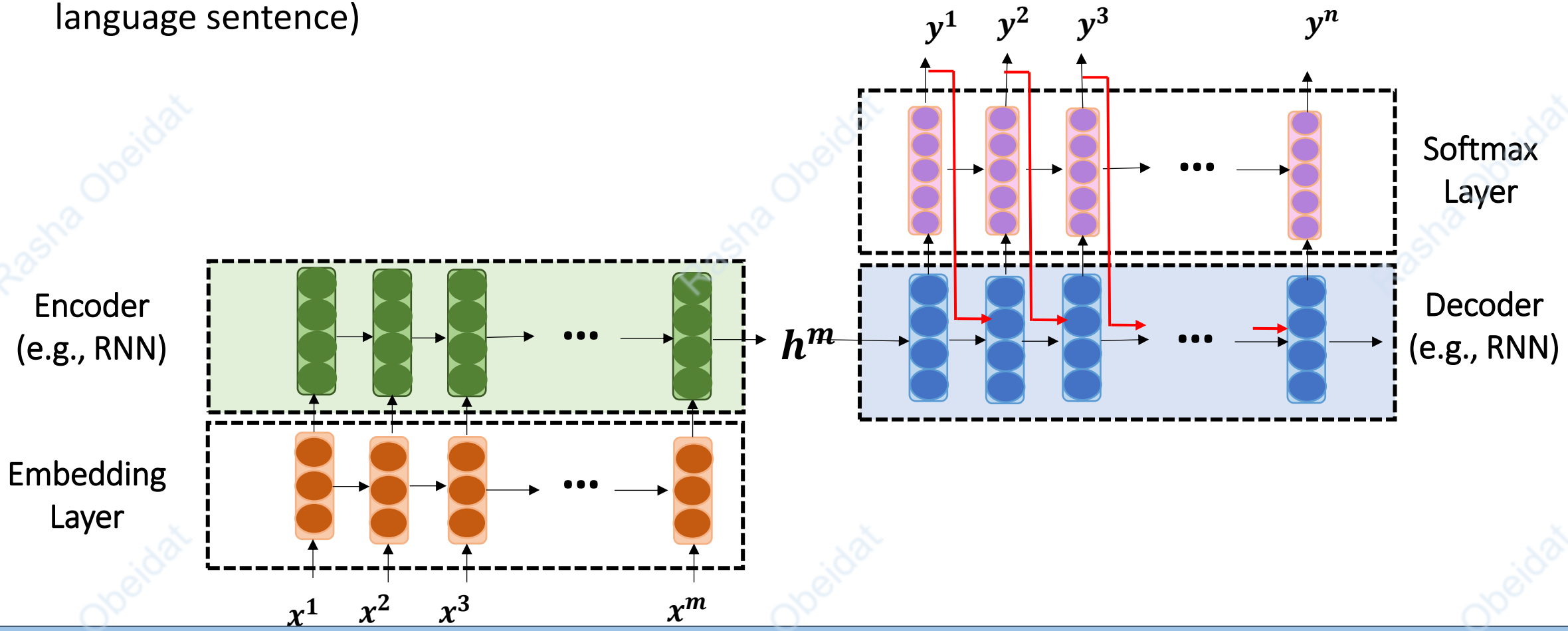
نعم.. امهلني بعض الوقت لو سمحت

Encoder-Decoder Architecture



Encoder-Decoder: Two Parts

Encoder: Processes the input sequence and encodes it into a fixed-size vector representation
Decoder: Decodes the vector representation and generates the output sequence (target language sentence)



Step1 : Input Sequence Encoding

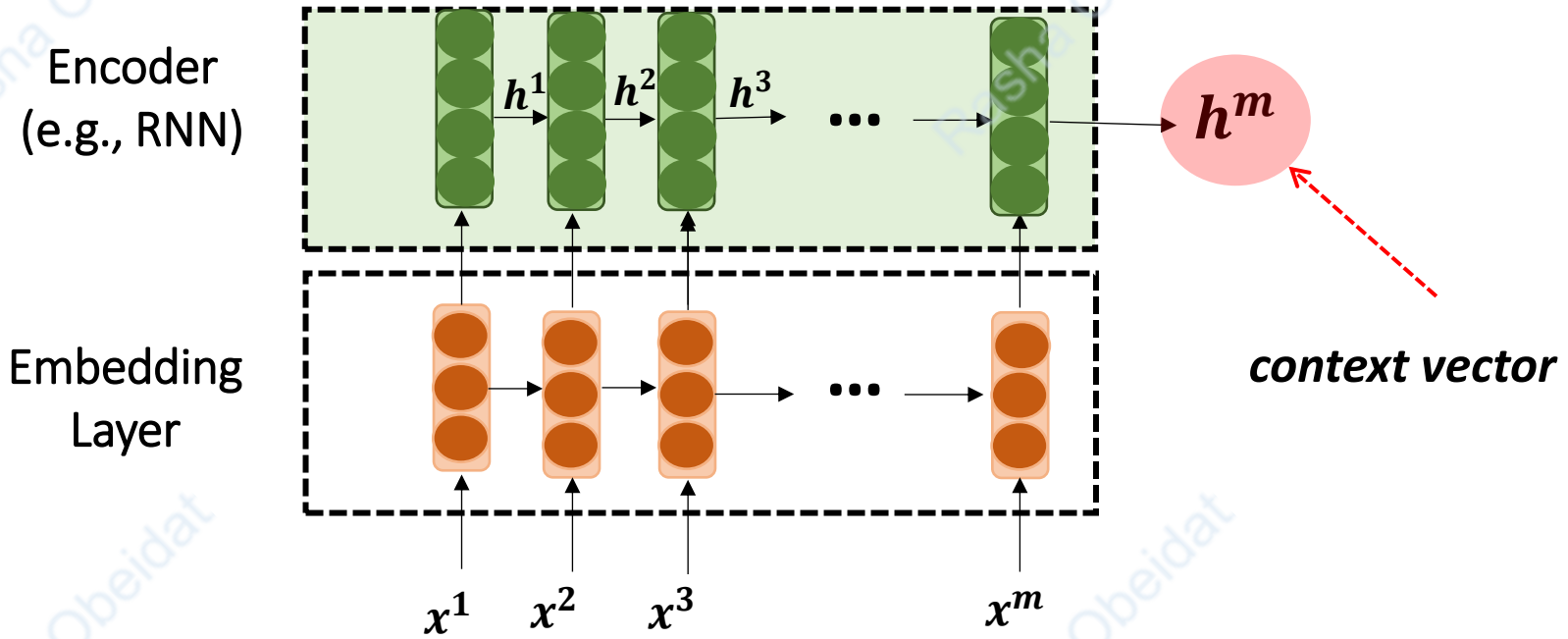
1. Tokenization: The input sequence "I am happy" is tokenized into a sequence of tokens: [I, am, happy].

2. Embedding: Each token is converted into a dense vector using an embedding layer.

3. Encoder: The token embeddings are fed into a recurrent neural network (RNN), such as an LSTM or GRU.

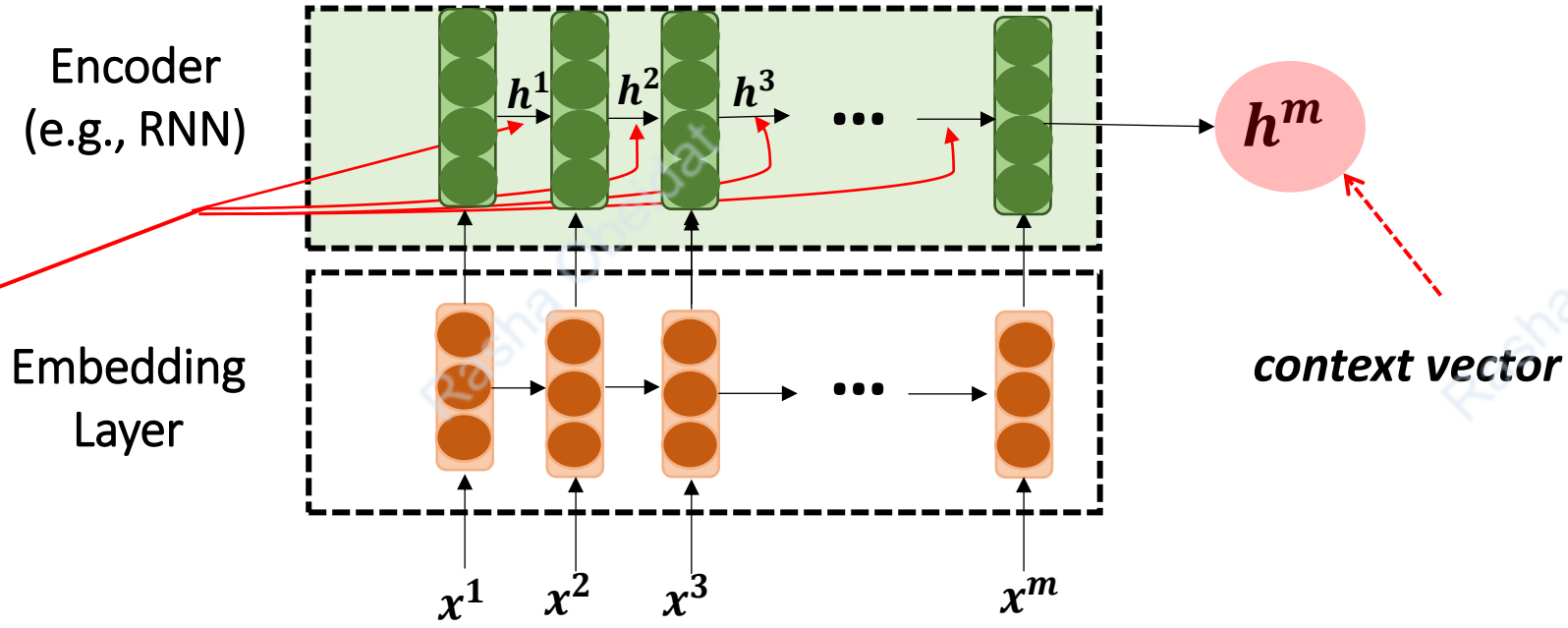
The encoder produces:

- A sequence of hidden states, one for each input token: $[h_1, h_2, h_3, \dots]$
- The final hidden state of the encoder($\boldsymbol{h_m}$ in the figure below), called a ***context vector*** and summarizes the entire input sequence.



Step1 : Input Sequence Encoding (Encoder)

All the intermediate states of the encoder are ignored, each state represents the part of the sentences sentence that has been processed so far.

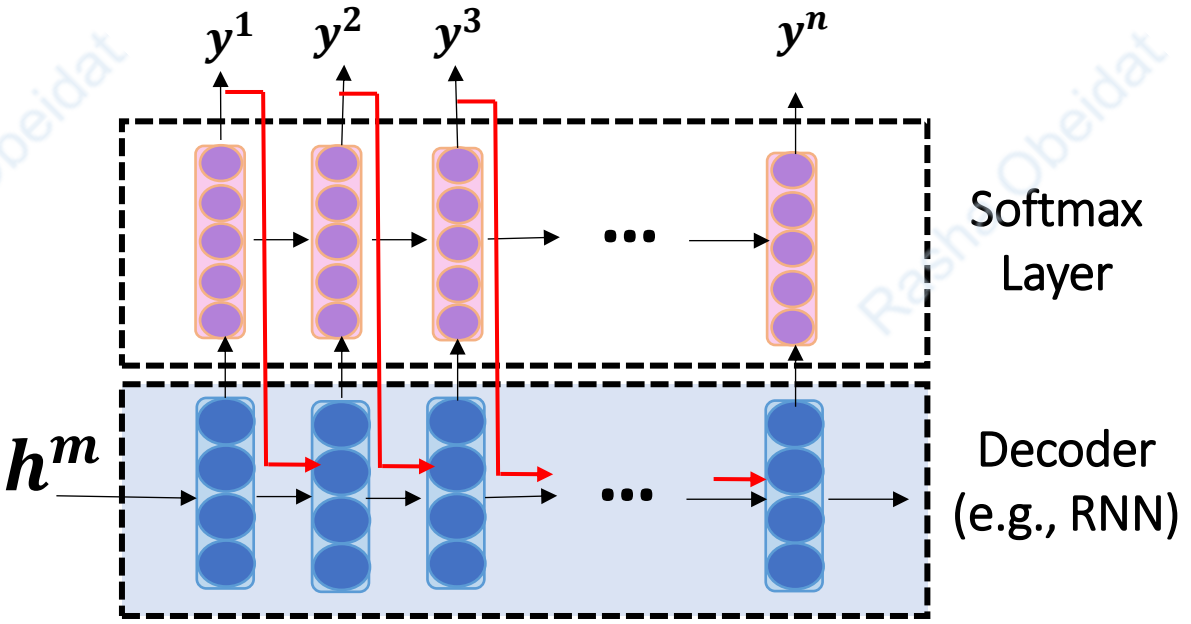


Step 2: Token-by-Token Decoding

Step2:

- The **Decoder** (which is also RNN/LSTM/GRU) starts from the context vector ($\boldsymbol{h_m}$) and trained to generate a sentence one token after another until a special token is generated.
- At each decoding step, the decoder predicts the next token based on:
  - Its previous hidden state.
  - The previously generated token.

Softmax layer is a softmax function that produces a probability distribution over Y.



Why Encoder-Decoder Architecture is Effective

- **Handles Variable-Length Data:** Suits tasks with differing input and output lengths (e.g., machine translation).
- **Learns Compact Representations:** Encodes input features into a meaningful vector for generating outputs.
- **Works Across Modalities:** Adapts to text, images, or speech tasks.
- **Leverages Attention:** Dynamically focuses on relevant input parts, improving output quality.
- **Enables End-to-End Training:** Optimizes all components simultaneously for better performance.
- **Adapts to Variants:** Allows flexibility with different implementations (e.g., LSTMs, Transformers).

Machine Translation

- **Machine Translation** is the task of automatically converting text in one source language to text in another language.
- Neural Machine Translation (NMT) is a way to perform MT with a single end-to-end neural network
- Enormous number of MT have been proposed in the literature, they can be divided into the following categories:
  - Phrase-based statistical MT
  - syntax-based statistical MT
  - Neural MT
  - Others
- Our focus is Neural MT

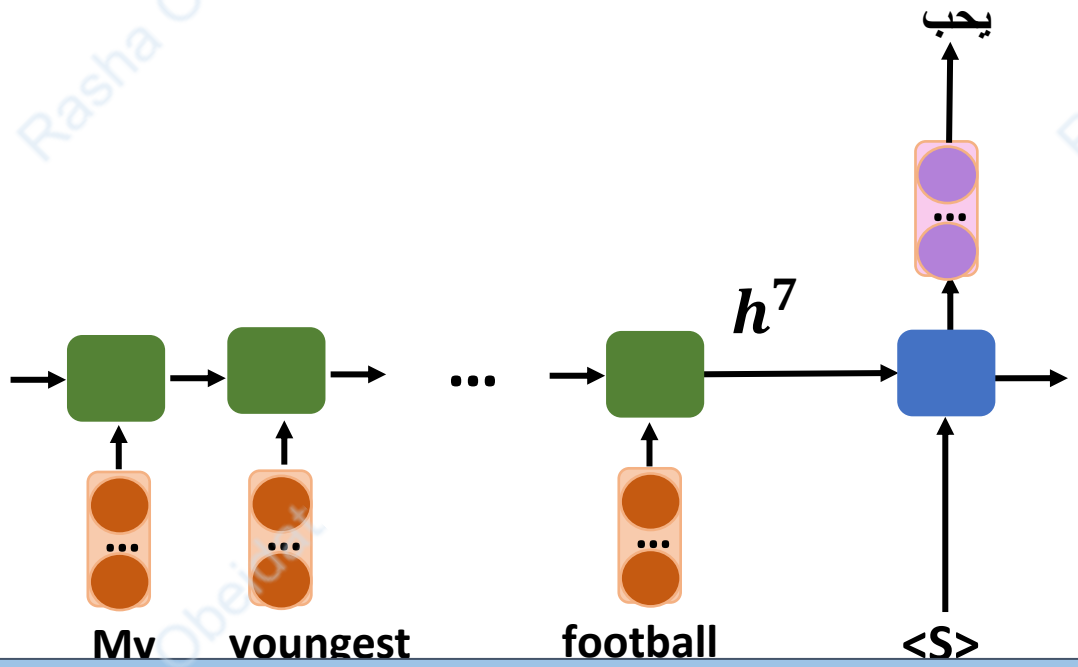
Seq2sec Application: Machine Translation

My youngest son likes to play football → <EOS> يحب ابني الصغير لعب كرة القدم <S>

x^1 x^2 x^3 x^4 x^5 x^6 x^7 y^8 y^7 y^6 y^5 y^4 y^3 y^2 y^1

<S> is a special tag indicating the begin of the sentence

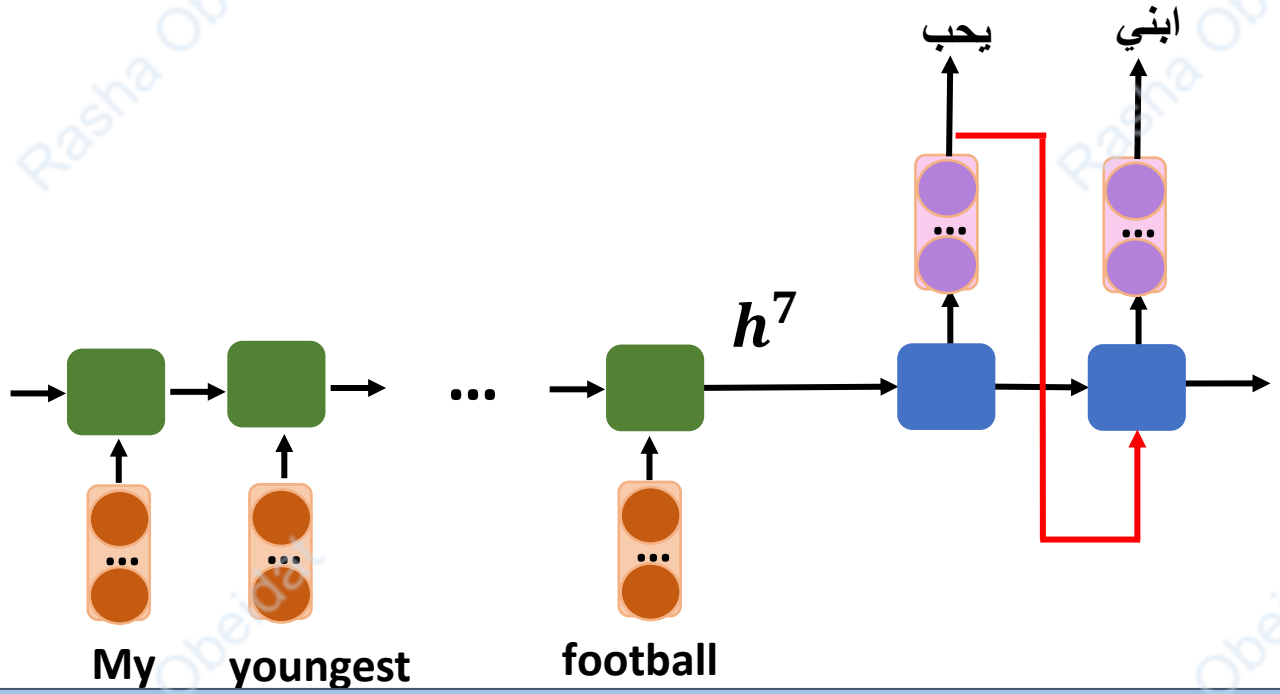
<EOS> is a special tag indicating the end of the sentence



Seq2sec Application: Machine Translation

My youngest son likes to play football → <EOS> يحب ابني الصغير لعب كرة القدم <S>

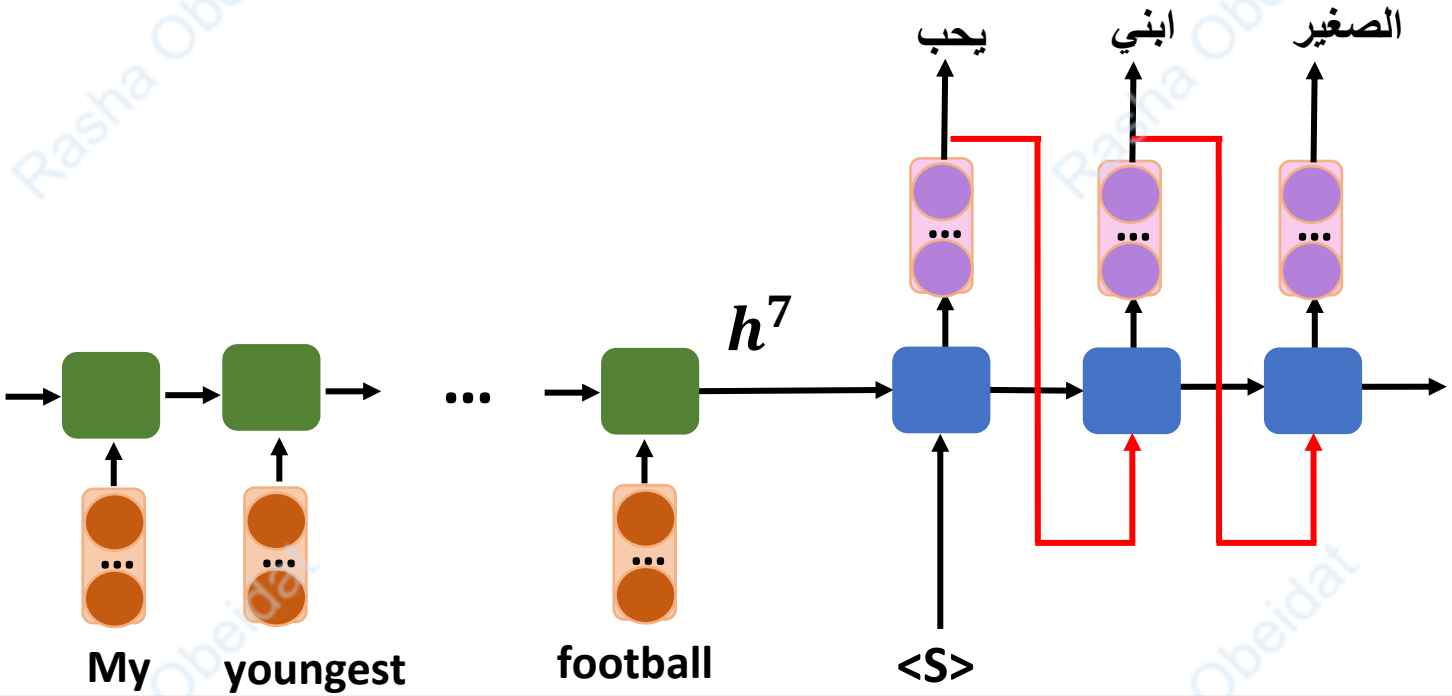
x^1 x^2 x^3 x^4 x^5 x^6 x^7 y^8 y^7 y^6 y^5 y^4 y^3 y^2 y^1



Seq2sec Application: Machine Translation

My youngest son likes to play football → <EOS> يحب ابني الصغير لعب كرة القدم <S>

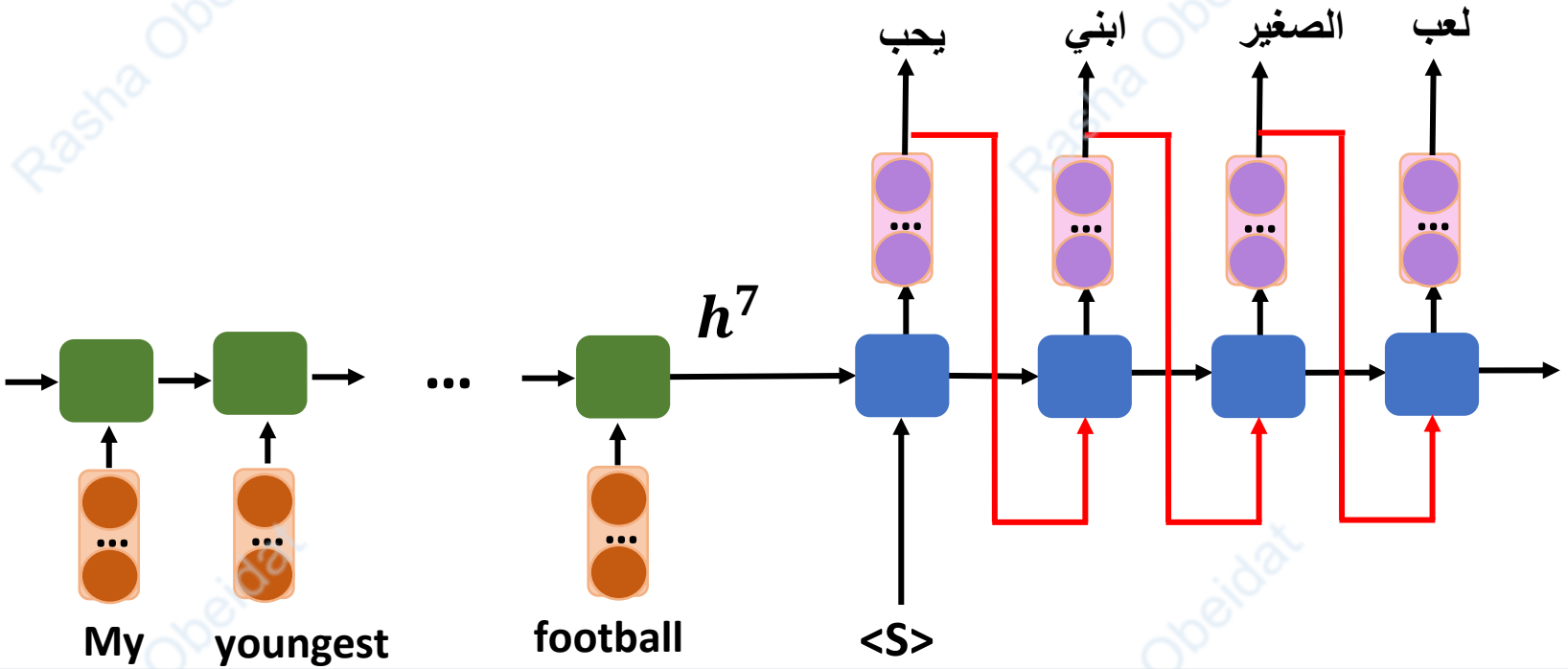
x^1 x^2 x^3 x^4 x^5 x^6 x^7 y^8 y^7 y^6 y^5 y^4 y^3 y^2 y^1



Seq2sec Application: Machine Translation

My youngest son likes to play football → <EOS> يحب ابني الصغير لعب كرة القدم <S>

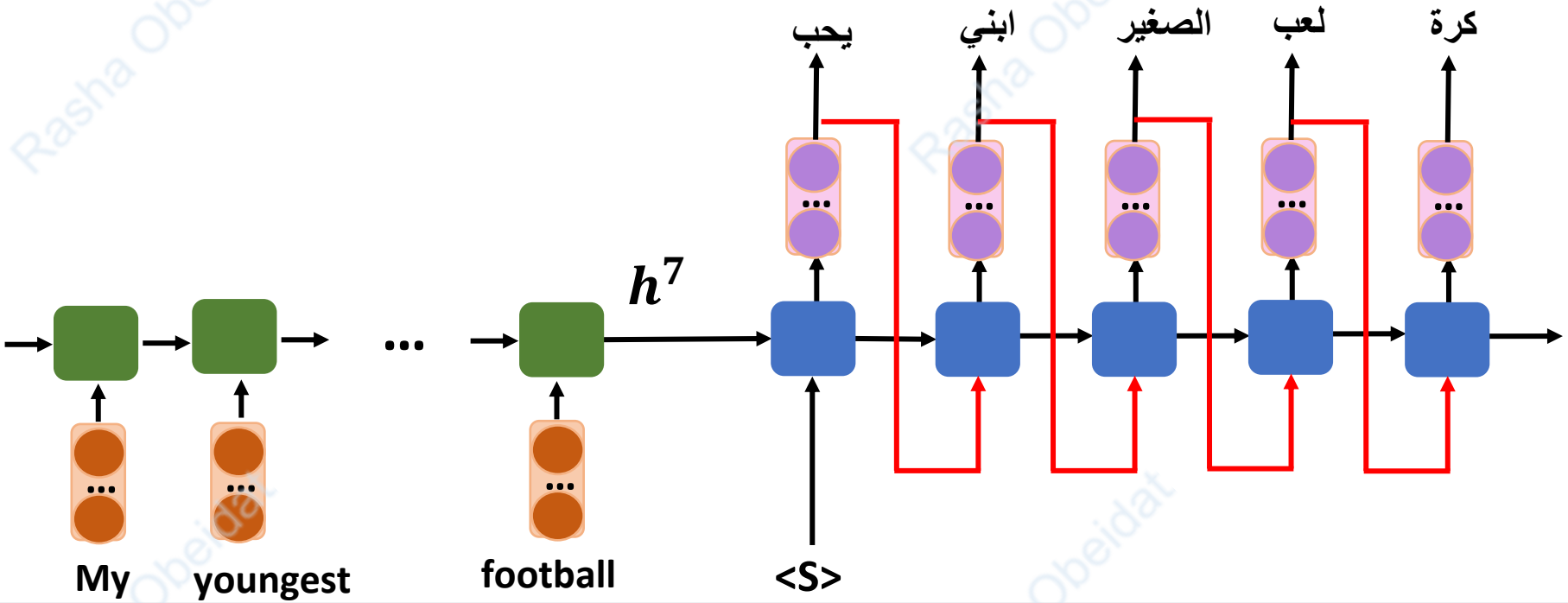
x^1 x^2 x^3 x^4 x^5 x^6 x^7 y^8 y^7 y^6 y^5 y^4 y^3 y^2 y^1



Seq2sec Application: Machine Translation

My youngest son likes to play football → <EOS> يحب ابني الصغير لعب كرة القدم <S>

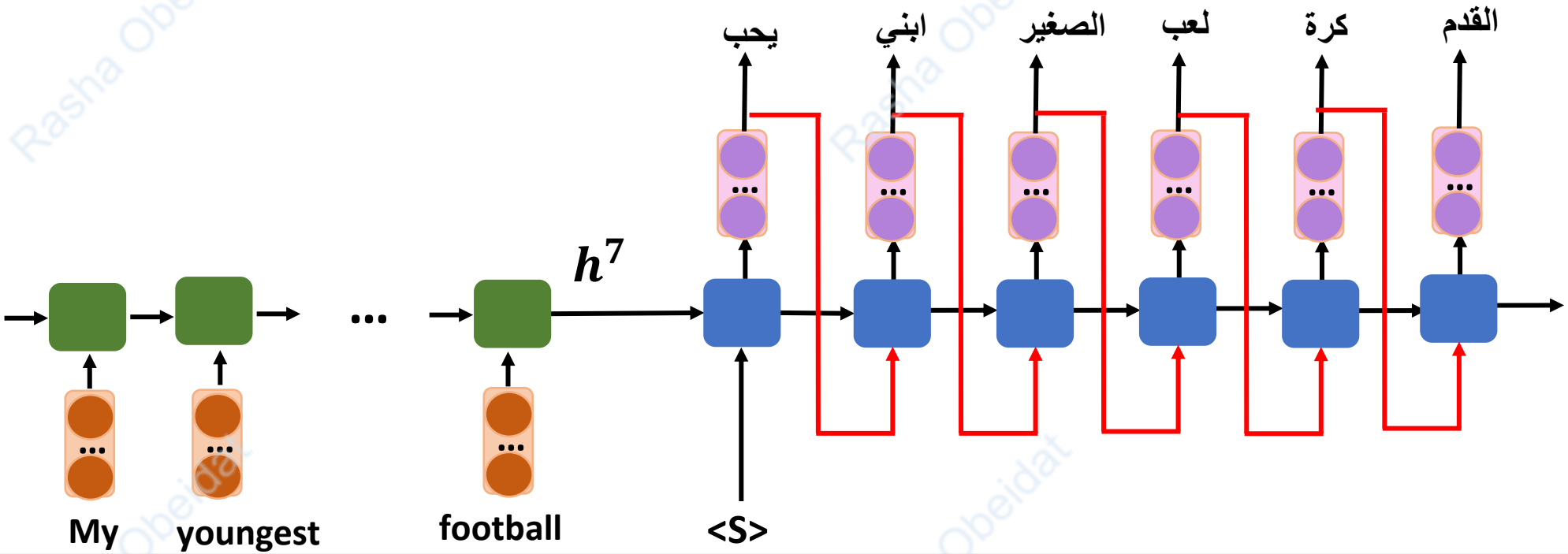
x^1 x^2 x^3 x^4 x^5 x^6 x^7 y^8 y^7 y^6 y^5 y^4 y^3 y^2 y^1



Seq2sec Application: Machine Translation

My youngest son likes to play football → <EOS> يحب ابني الصغير لعب كرة القدم <S>

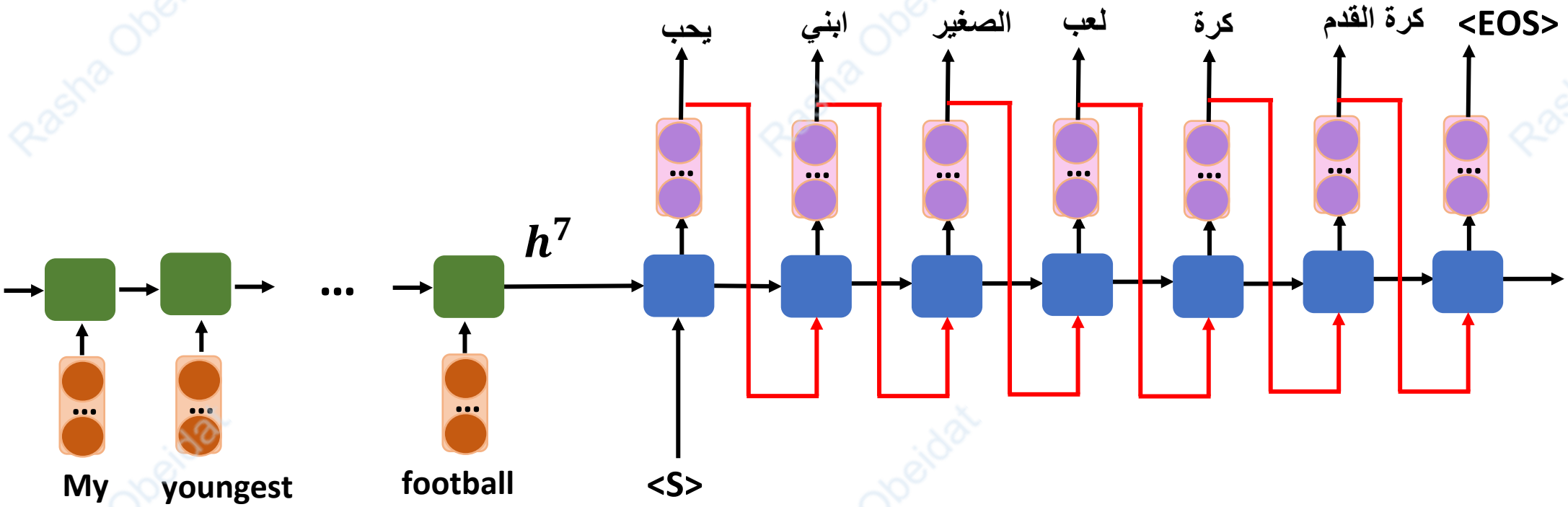
x^1 x^2 x^3 x^4 x^5 x^6 x^7 y^8 y^7 y^6 y^5 y^4 y^3 y^2 y^1



Seq2sec Application: Machine Translation

My youngest son likes to play football → <EOS> كرة القدم لعب الصغير ابني يحب <s>

x^1 x^2 x^3 x^4 x^5 x^6 x^7 y^8 y^7 y^6 y^5 y^4 y^3 y^2 y^1



Decoder Output as Input for the Next Time Step

- **Initial Step (First Word):**
  - For the first timestep **(t=0),** where no previous word exists, a special start token (**<S>)** is used as the input.
- **Starting from time step t=1,**
  - **At Training Phase (Teacher Forcing):** the decoder is provided with the ground-truth token **(the embedding of the word)** from the target sequence.
  - **At Inference Phase (Autoregressive Decoding):**
    - **Greedy Decoding (Argmax):** The word with the highest probability is selected as the next token, and its embedding is used as the input for the next timestep.
    - **Beam Search:** The embeddings of the top b tokens (highest cumulative probabilities) are used as inputs to explore multiple possible sequences in parallel.

Training the seq2sec model

- The training set consists of pairs of sequences (It is possible the input sequence could have the length one).
- The model is trained to process the input sequences and force to generate the correct output sequence, I.e., assign high probability to the output sequence.
- The lower the probability assigned to the correct output sequence the more the model is panelized
- The Loss (e.g., cross-entropy loss) is computed for each token in the sequence. The individual loss values summed up to give the total loss.
- All model parameters (possibly including the word embeddings) are adjusted by back-propagating the total loss through time (**BPTT**) using an (gradient descent)-based optimization technique.

Inference

- In MT, inference means choosing the most probable translation from the decoder outputs (most likely the output sequence) given the input sequence, i.e., $P(y^1, y^2, \dots, y^n | x^1, x^2, \dots, x^m)$
- Training MT seq2seq model can be considered as modeling $P(y^1, y^2, \dots, y^n | x^1, x^2, \dots, x^m; \theta)$ where θ is the model parameters.
- At a time step t, the decoder tries to compute the probability of every word in the vocabulary given the input sentence and the previous output. I.e. $p(y^t | x^1, \dots, x^m, y^{t-1}; \theta)$.

$$P(y|x) = P(y^1, y^2, \dots, y^n | x^1, x^2, \dots, x^m; \theta)$$
$$= \prod_{t=1}^{n} p(y^t | x^1, \dots, x^m, y^1, \dots, y^{t-1}; \theta)$$

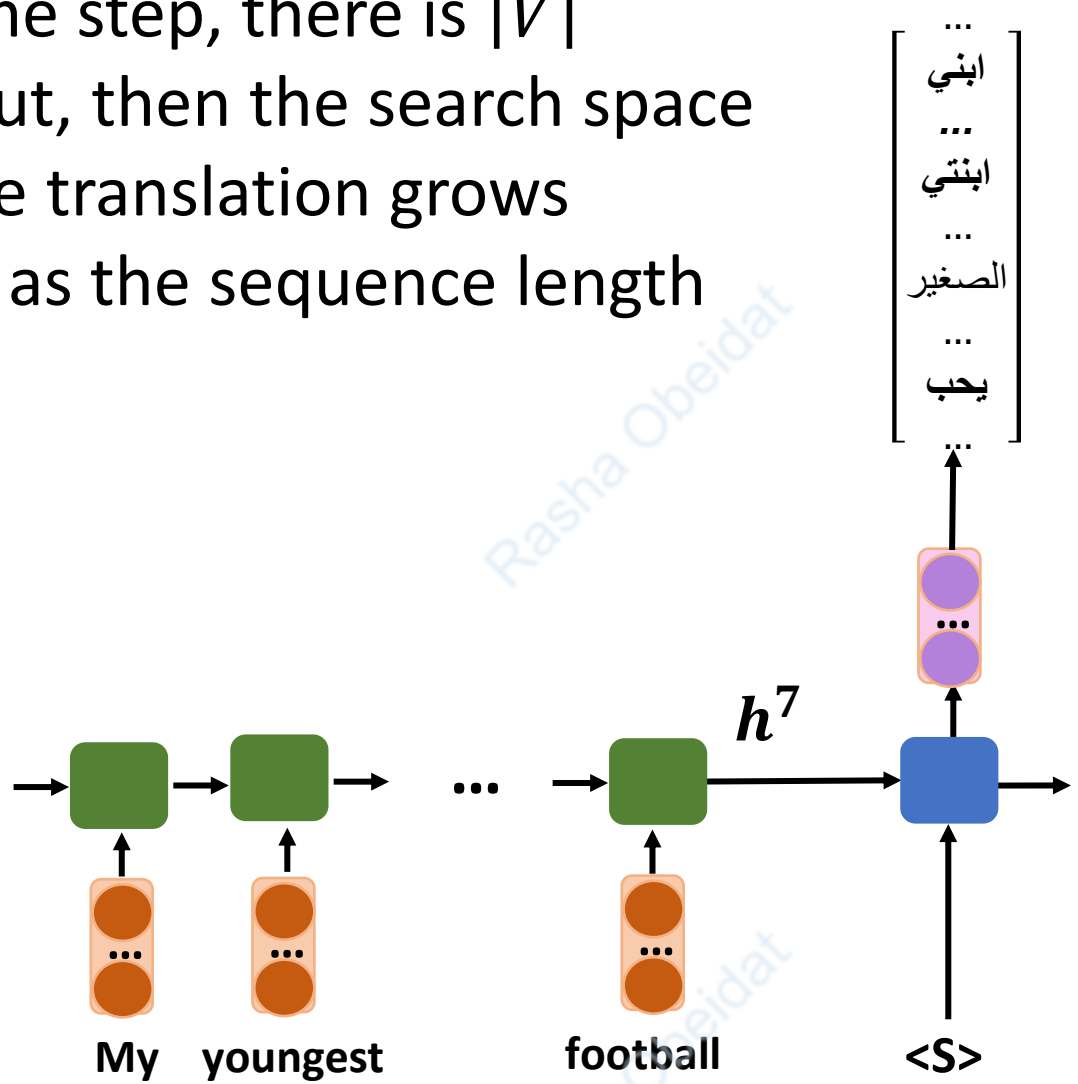
Seq2Seq is Conditional Language Model

- The sequence-to-sequence model is an example of a ***Conditional Language Model***
  - Language Model because the decoder is predicting the next word of the target sentence.
  - Conditional because its predictions are also conditioned on the source sentence x.
- NMT directly calculates $P(y|x)$:

$$P(y|x) = P(y^1, y^2, \dots, y^n | x^1, x^2, \dots, x^m; \theta)$$
$$= \prod_{t=1}^{n} p(y^t | x^1, \dots, x^m, y^1, \dots, y^{t-1}; \theta)$$

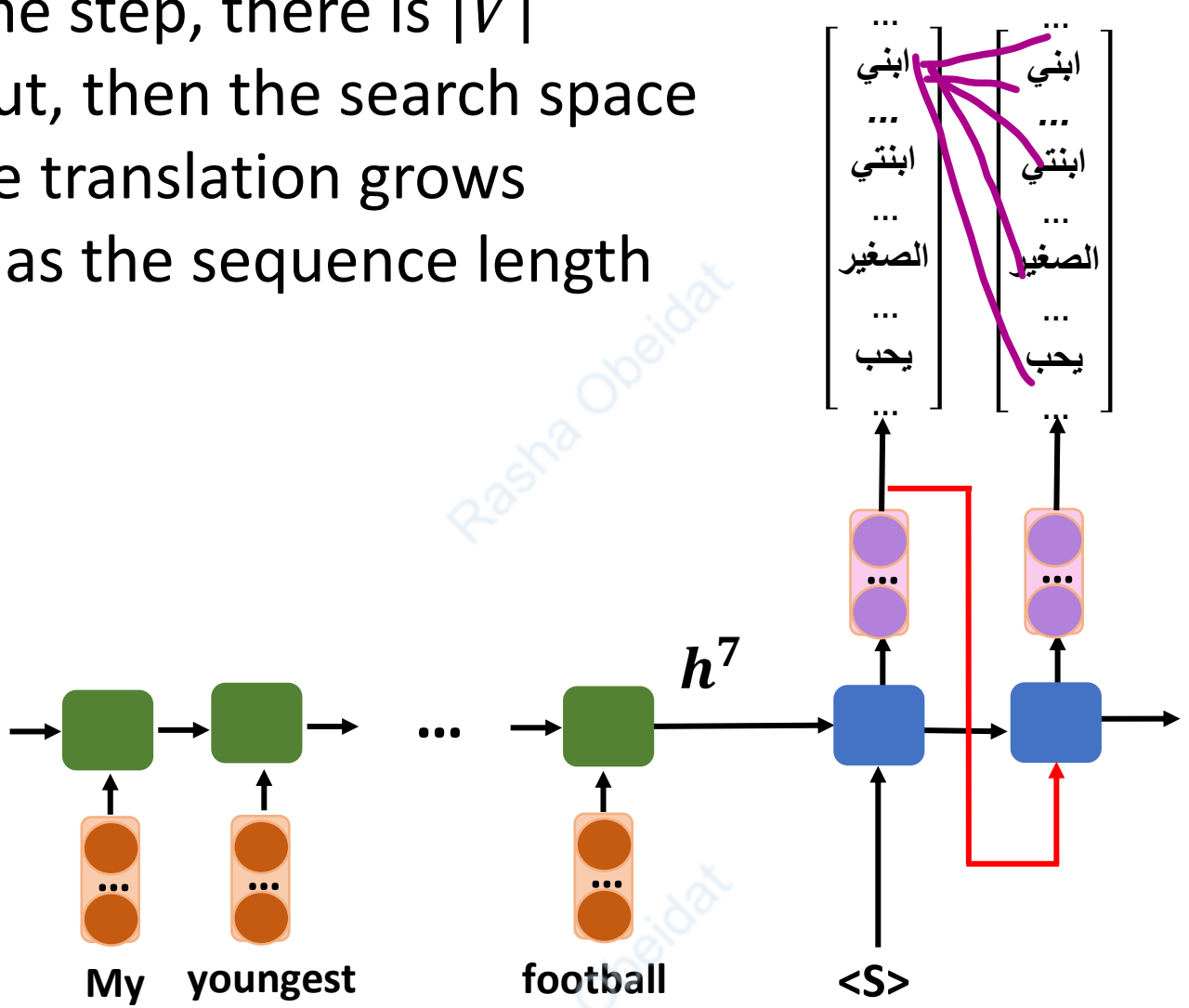
Inference

- Since each time step, there is $|V|$ possible output, then the search space of the possible translation grows exponentially as the sequence length grows.



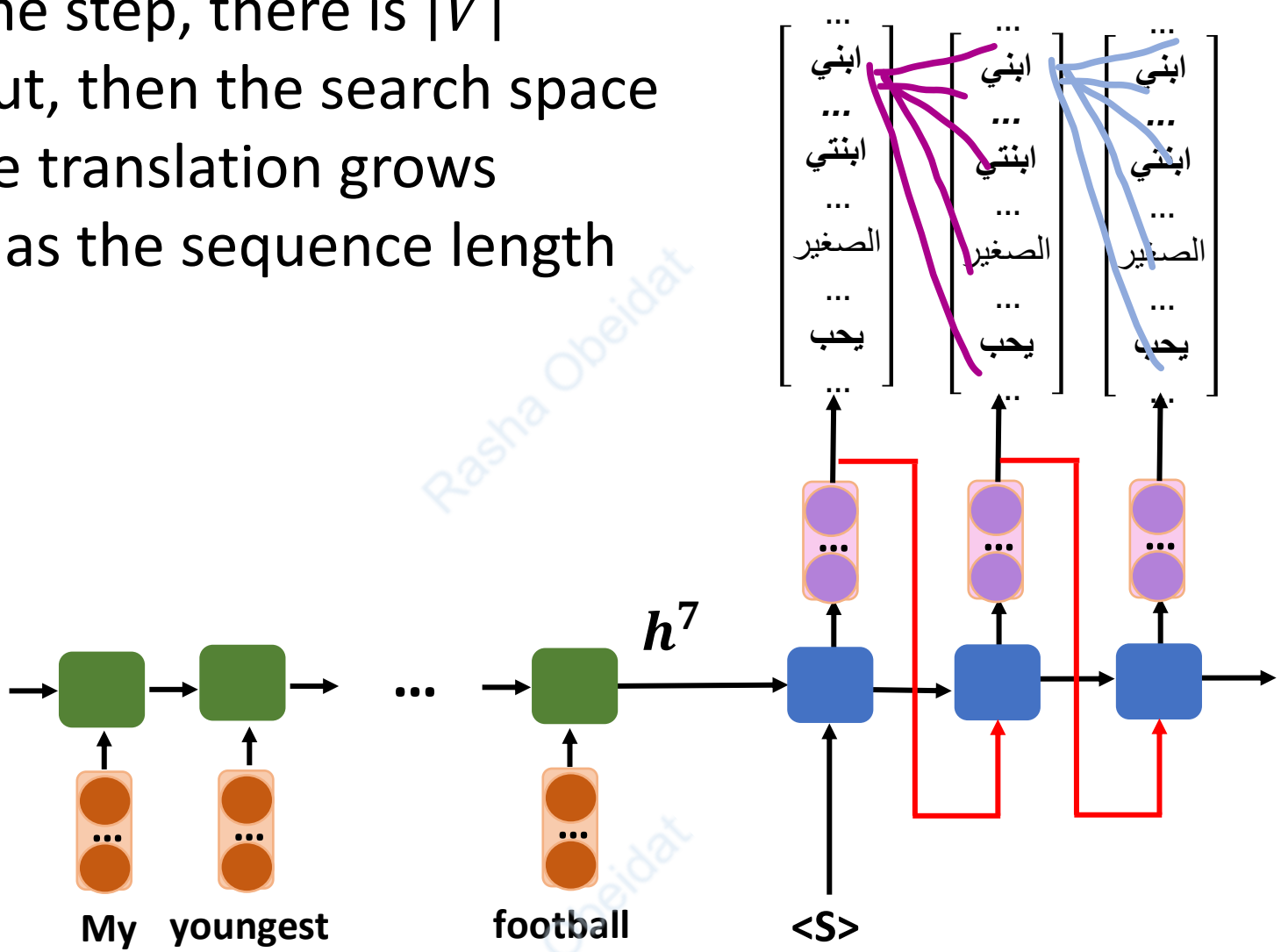
Inference

- Since each time step, there is $|V|$ possible output, then the search space of the possible translation grows exponentially as the sequence length grows.



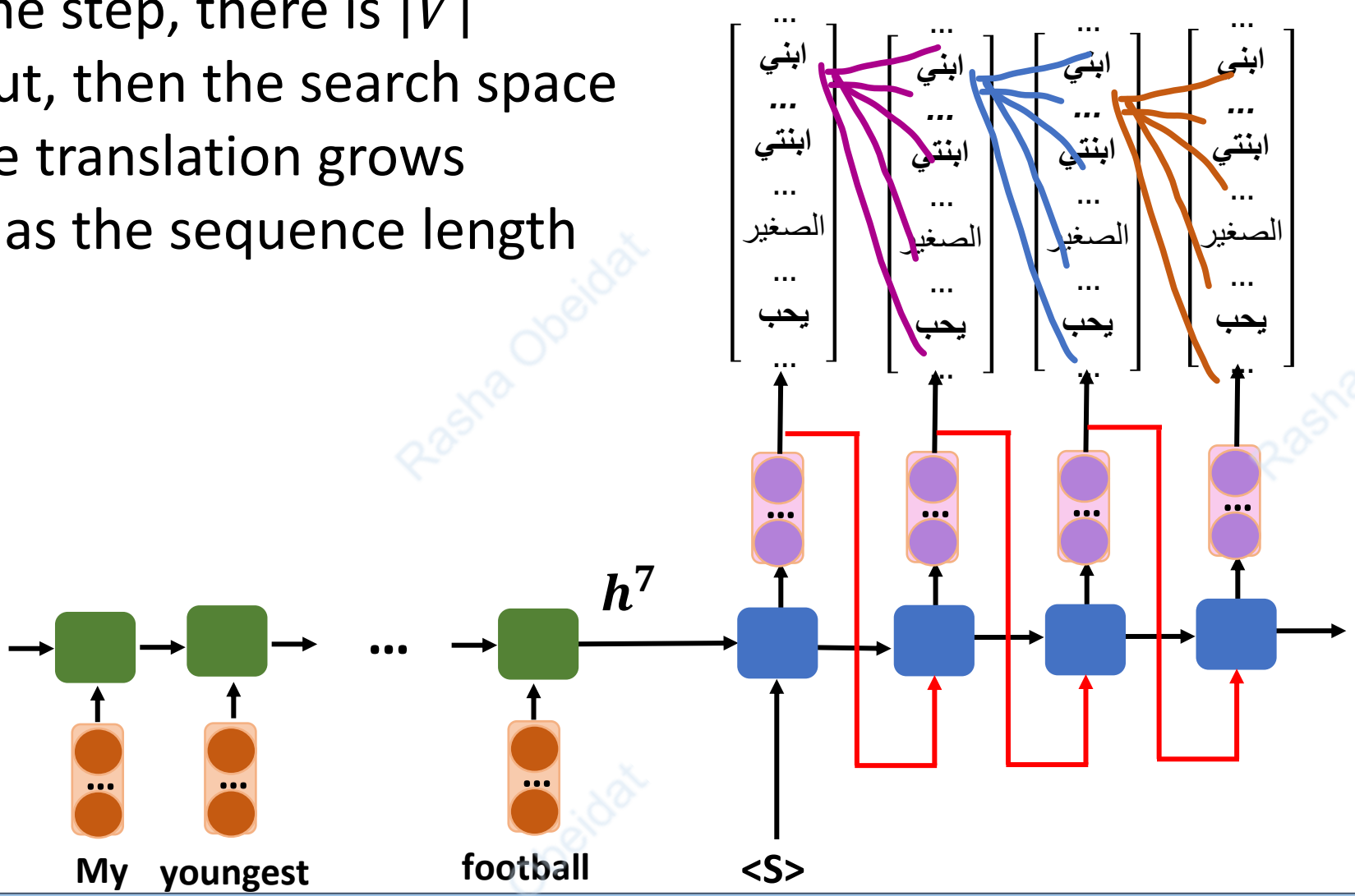
Inference

- Since each time step, there is $|V|$ possible output, then the search space of the possible translation grows exponentially as the sequence length grows.



Inference

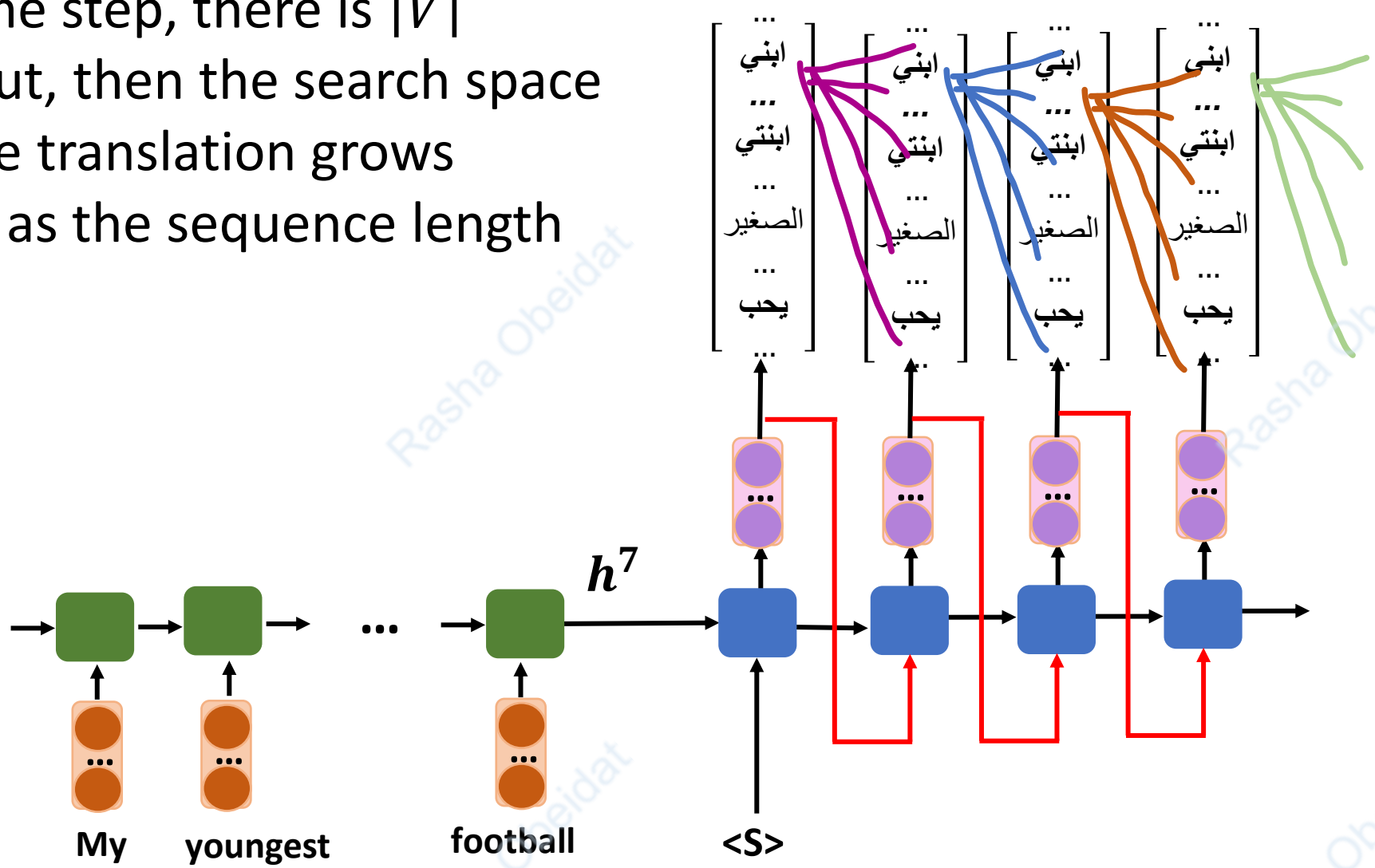
- Since each time step, there is $|V|$ possible output, then the search space of the possible translation grows exponentially as the sequence length grows.



Inference

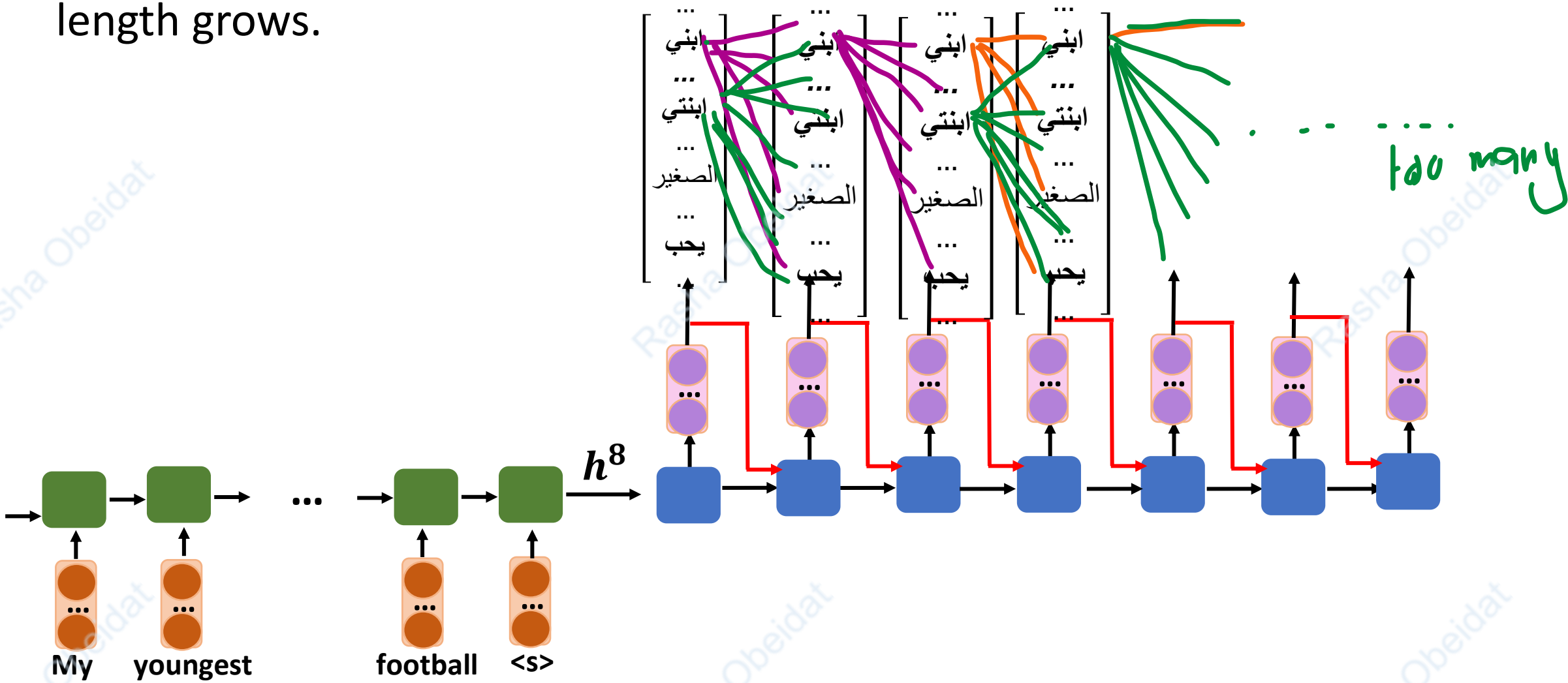
Too Many Combinations

- Since each time step, there is $|V|$ possible output, then the search space of the possible translation grows exponentially as the sequence length grows.



Inference

- Since each time step, there is $|V|$ possible output, then the search space of the possible translation grows exponentially as the sequence length grows.



Inference

- **Brute force** : infeasible at all , waaaay too many options!

Rasha Obeidat

Inference

- **Brute force** : infeasible at all , waaaay too many options!
- **Greedy**: get the word with the highest probability at each time step.
  - Computationally efficient but may produce poor translation!

Rasha Obeidat

Greedy search

My youngest son likes to play football → <EOS> يحب ابني الصغير لعب كرة القدم <S>

Distribution over the entire vocabulary

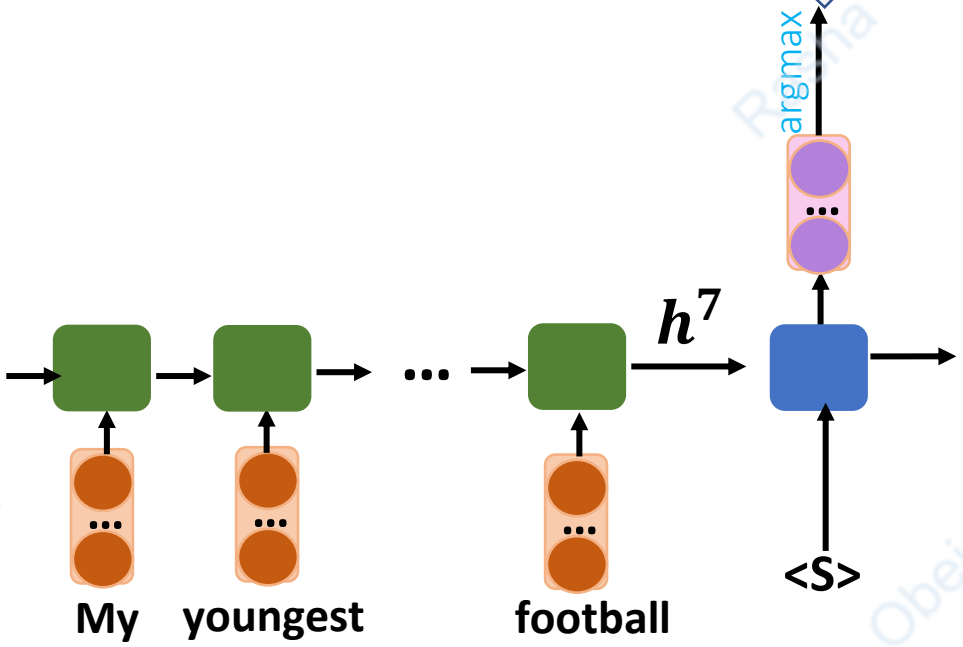
$p(\text{يحب}|x) = \mathbf{0.61}$

$p(\text{ابني}|x) = 0.099999$

$p(\text{العلم}|x,) = 0.000001$

...

$\underset{y}{\text{argmax}}\, p(y^1|x;\theta) =$ يحب

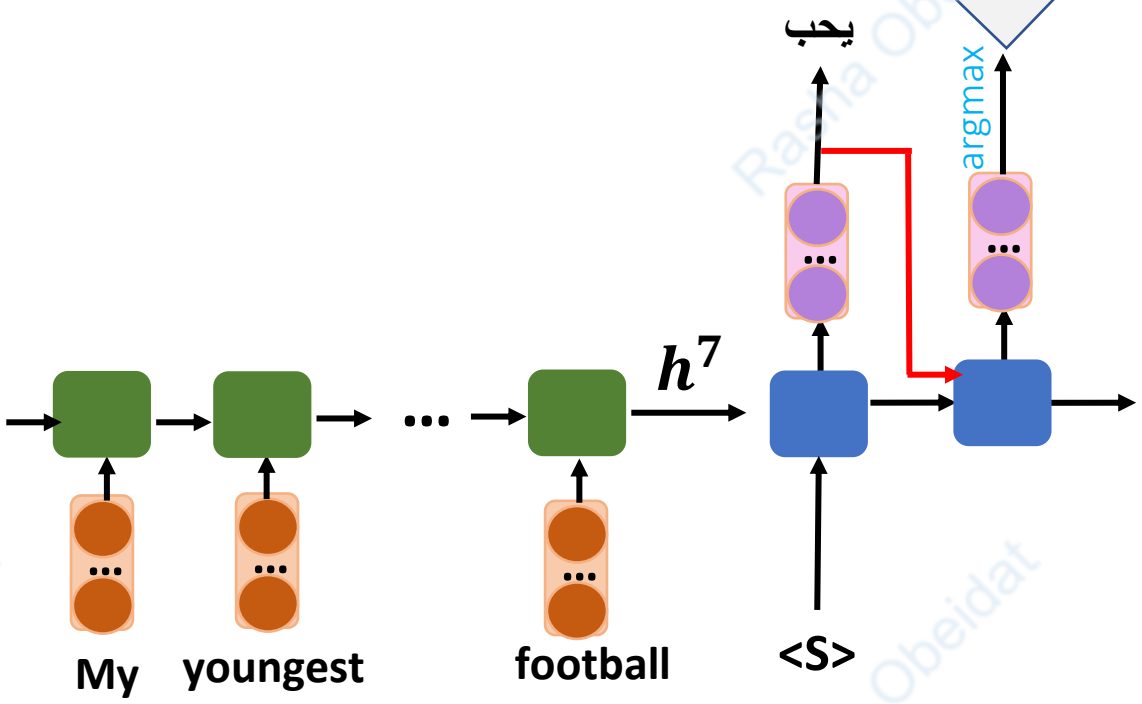


Greedy search

My youngest son likes to play football → <EOS> يحب ابني الصغير لعب كرة القدم <S>

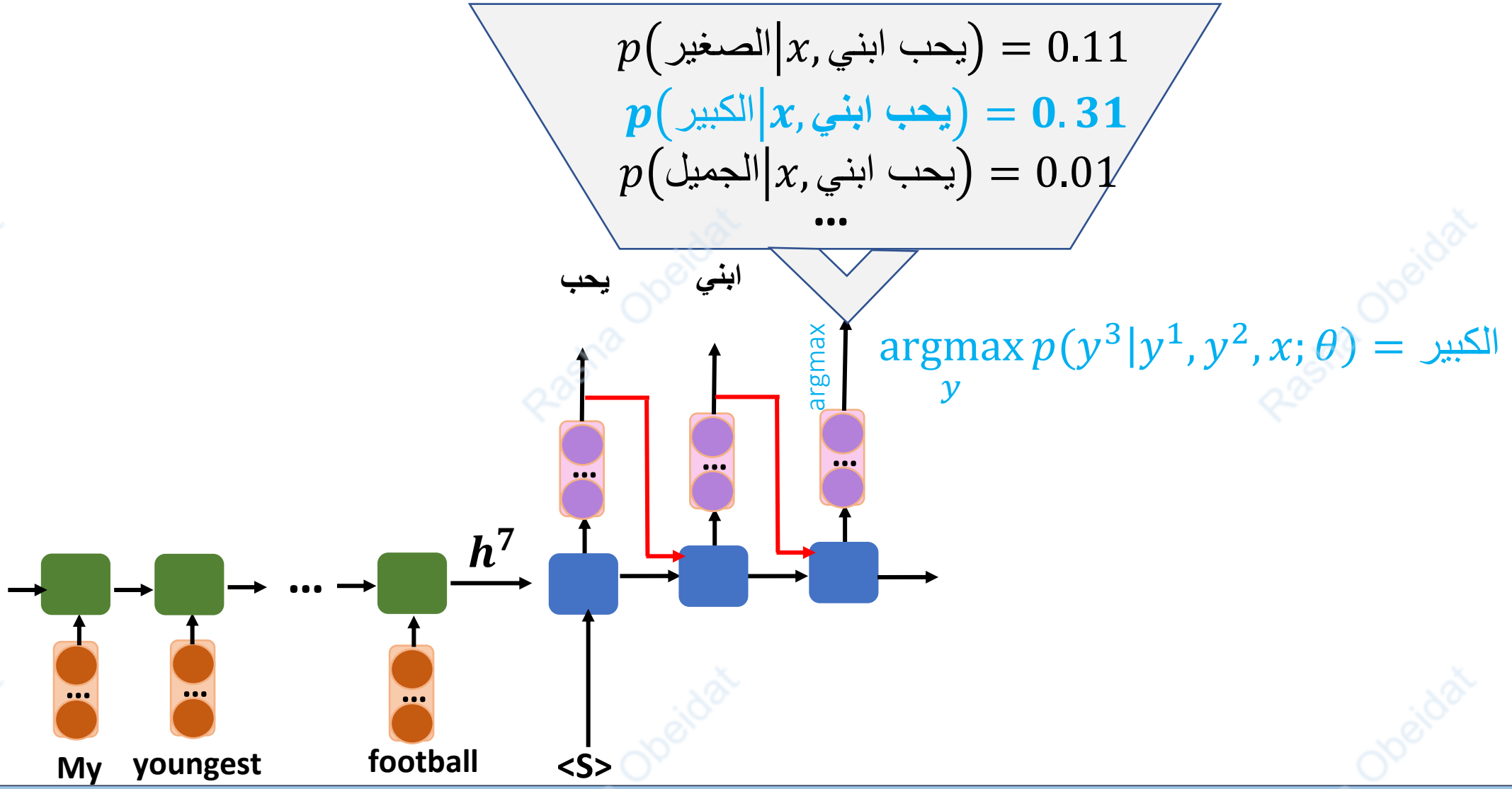
$$p(\text{ابني} | x, \text{يحب}) = \mathbf{0.31}$$
$$p(\text{الولد} | x, \text{يحب}) = 0.23$$
$$p(\text{يقول} | x, \text{يحب}) = 0.000001$$
$$\dots$$

$$\underset{y}{\text{argmax}}\, p(y^2 | y^1, x; \theta) = \text{ابني}$$



Greedy search: When to stop?

My youngest son likes to play football → <EOS> يحب ابني الصغير لعب كرة القدم <S>



Greedy search: When to stop?

- The process continues until the model predicts the **<EOS>** token. In other words, the generation ends at the step ***where <EOS> has the highest probability***, given the input and the sequence of outputs generated so far.

Rasha Obeidat

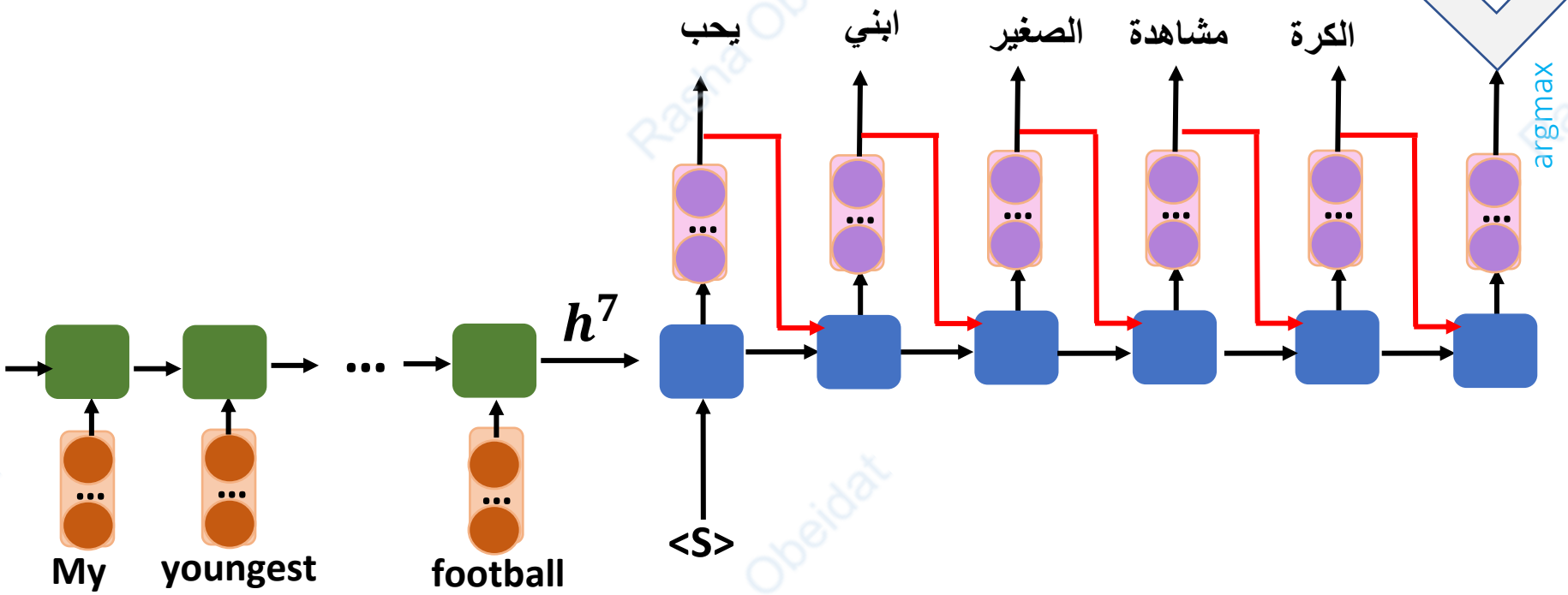
Greedy search

My youngest son likes to play football → <EOS> يحب ابني الصغير لعب كرة القدم <S>

y^1-y^5 : يحب ابني الصغير مشاهدة الكرة

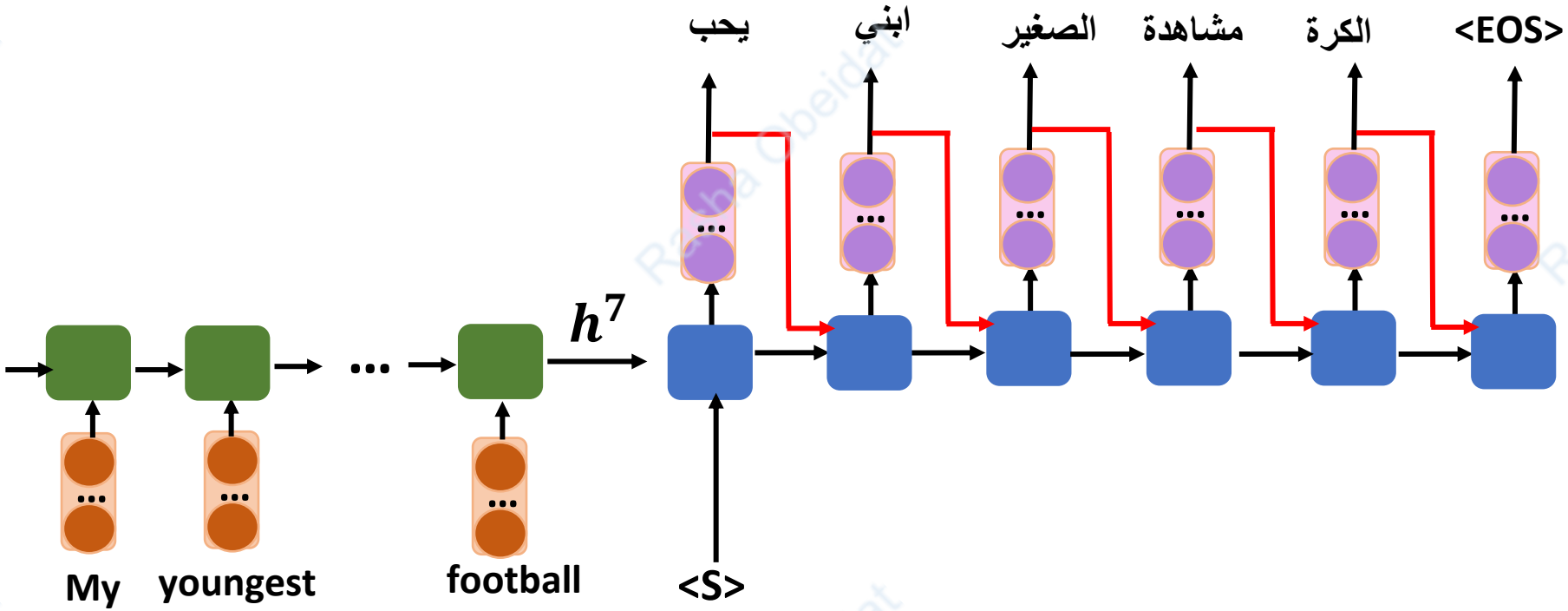
$$\underset{y}{\operatorname{argmax}}\, p(y^6|, y^1\text{–}y^5, x; \theta) = \text{<EOS>}$$

$$p(\text{لانه}|x, y^1\text{–}y^5) = 0.11$$
$$\boldsymbol{p(\{[EOS]|x, y^1\text{–}y^5) = 0.43}$$
$$p(\text{دائما}|x, y^1\text{–}y^5) = 0.01$$
...



Greedy search

My youngest son likes to play football → <EOS> الكرة مشاهدة الصغير ابني يحب <S>



Inference

- **Brute force** : infeasible at all , waaaay too many options!
- **Greedy**: get the word with the highest probability at each time step.
  - Computationally efficient but may produce poor translation!

Solution ?

Using an approximation search algorithm such as

beam search!

Rasha Obeidat

What is Beam Search?

- A **search algorithm** used in sequence-to-sequence models (e.g., machine translation, text generation) to balance between ***exploration*** and ***exploitation***.
  - **Exploration**: Explore additional high-probability paths that might lead to the correct translation.
  - **Exploitation**: Focus on the already explored highly probable paths in the search space to progress toward generating the translation.
- Keeps track of the top b sequences (beams) at each step, rather than just the single most likely one.
- Key Features:
  - **Beam Width (b):** Number of sequences (hypotheses) retained at each decoding step.
  - **Cumulative Scores**: Ranks sequences based on the total probability of the sequence so far.

How Beam Search Works:

- Start with the special token <S>.
- At each timestep:
    1. Expand each sequence in the beam with all possible next tokens.
    2. Compute the cumulative probabilities for all candidate sequences.
    3. Retain the top b sequences.
- Stop when all top b sequences (*all hypotheses in the beam)* reach <EOS> or a maximum length.

Beam search

My youngest son likes to play football <s> → <s> يحب ابني الصغير لعب كرة القدم

$|V|$

$$\begin{bmatrix} \dots \\ \text{اب} \\ \dots \\ \text{ابني} \\ \dots \\ \text{احب} \\ \dots \\ \text{يحب} \\ \dots \\ \text{يفضل} \\ \dots \end{bmatrix}$$

Vocabulary V

$$P\left(\begin{bmatrix} \text{احب} \\ \text{يحب} \\ \text{يفضل} \\ \dots \\ \text{اب} \\ \text{ابني} \\ \dots \\ \text{ابنتي} \\ \dots \\ \text{يد} \\ \dots \end{bmatrix} | x; \theta\right) = \begin{bmatrix} 0.214 \\ 0.198 \\ 0.175 \\ \dots \\ 0.003 \\ 0.002 \\ \dots \\ 0.0011 \\ \dots \\ 0.0003 \\ \dots \end{bmatrix}$$

The probabilities produced by **Decoder** at time ***step*** 1 (sorted)

$b = 3$

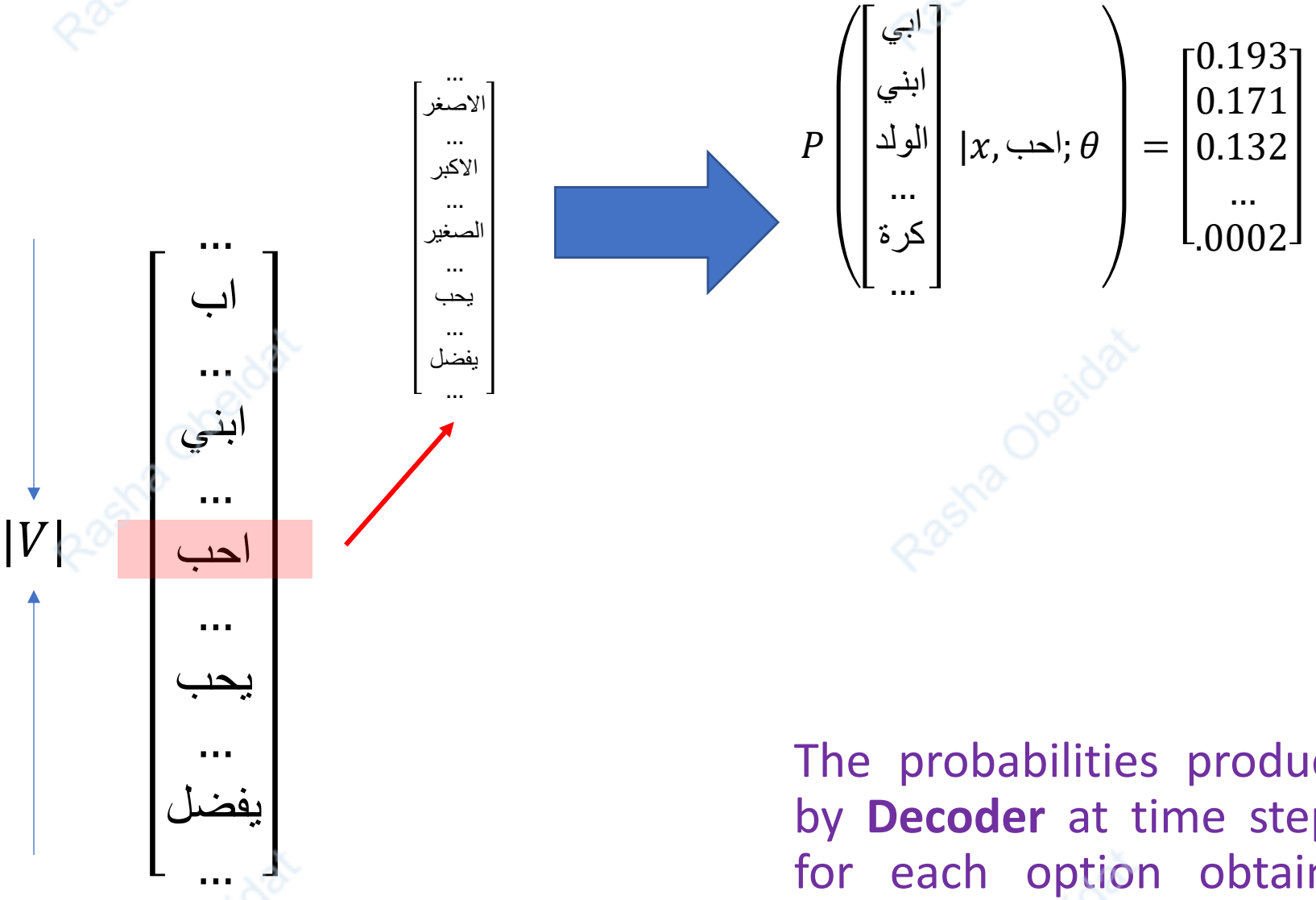
$$\begin{bmatrix} \text{احب} \rightarrow 0.214 \\ \text{يحب} \rightarrow 0.198 \\ \text{يفضل} \rightarrow 0.172 \end{bmatrix}$$

The most probable 3 tokens

My youngest son likes to play football <s> → <s> يحب ابني الصغير لعب كرة القدم

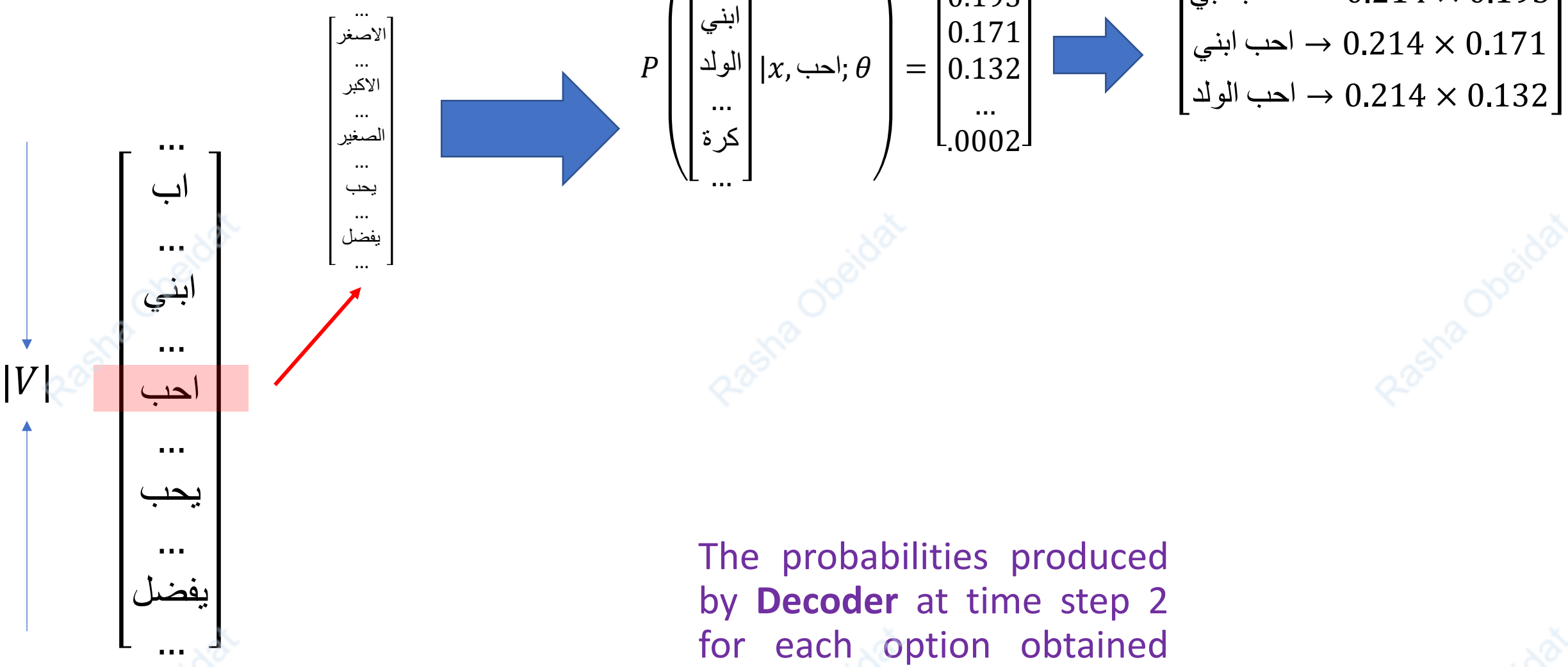


My youngest son likes to play football <s> → <s> يحب ابني الصغير لعب كرة القدم



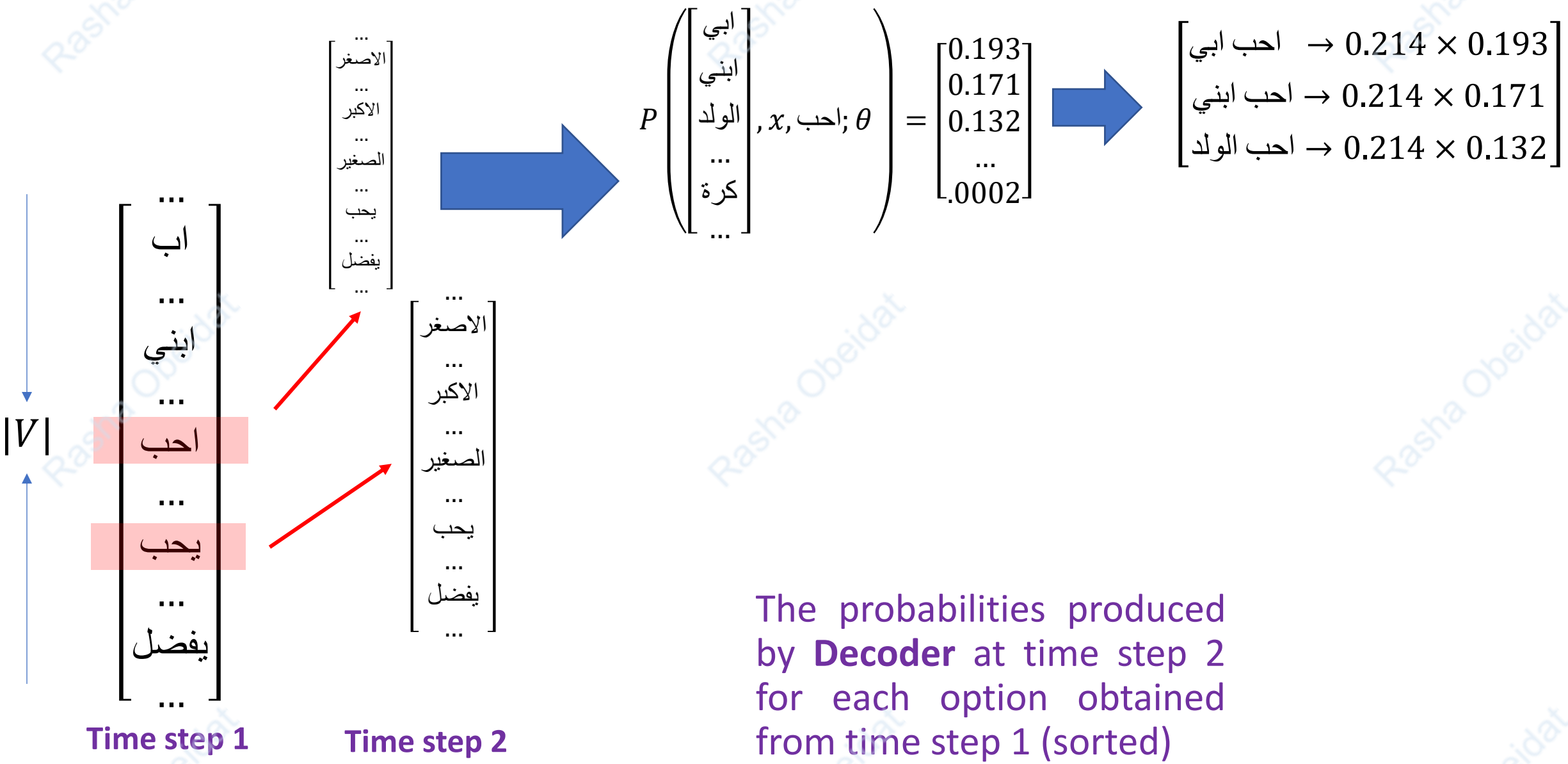
The probabilities produced by **Decoder** at time step 2 for each option obtained from time step 1 (sorted)

My youngest son likes to play football <s> → <s> يحب ابني الصغير لعب كرة القدم



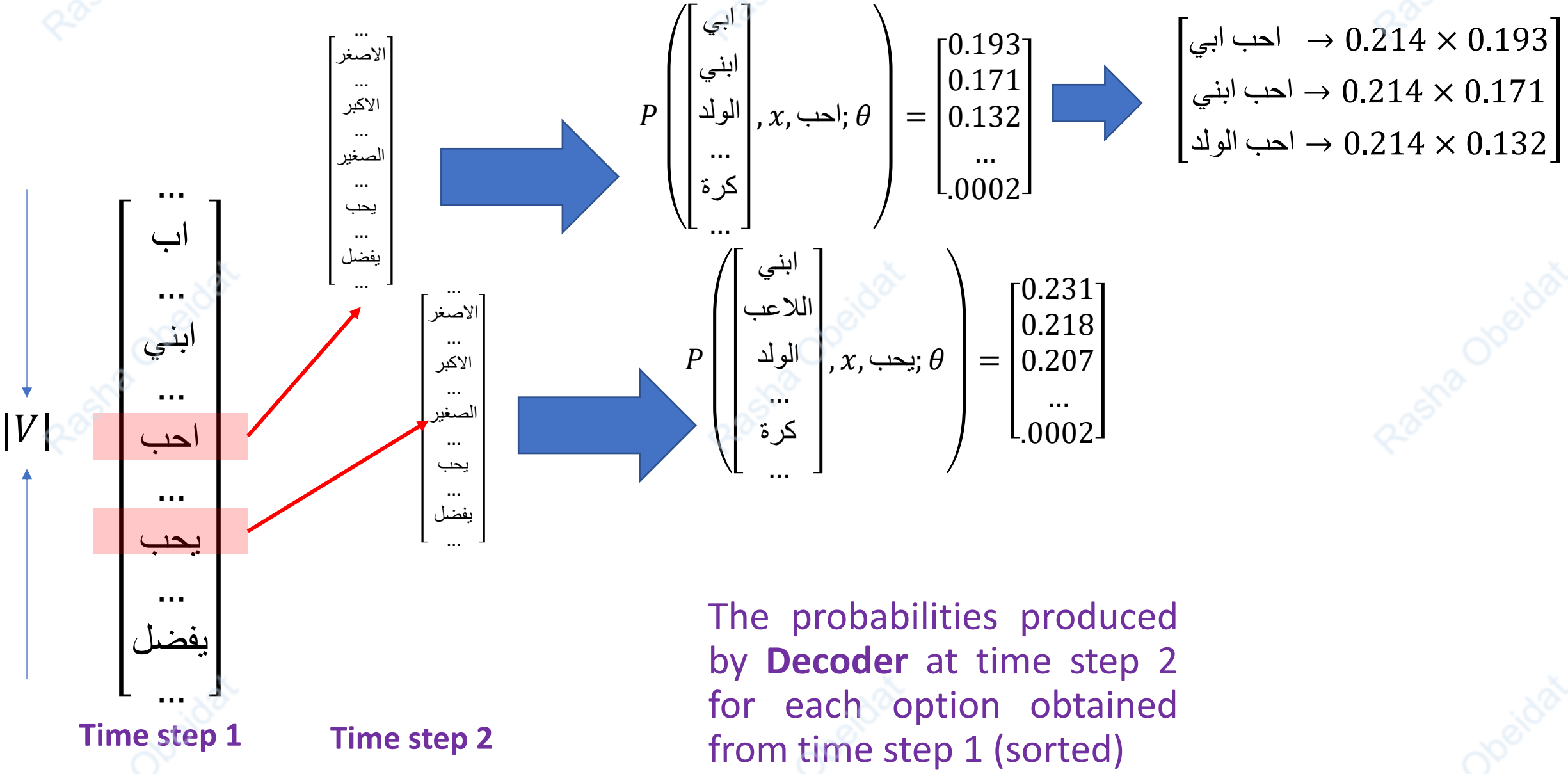
The probabilities produced by **Decoder** at time step 2 for each option obtained from time step 1 (sorted)

My youngest son likes to play football <s> → <s> يحب ابني الصغير لعب كرة القدم



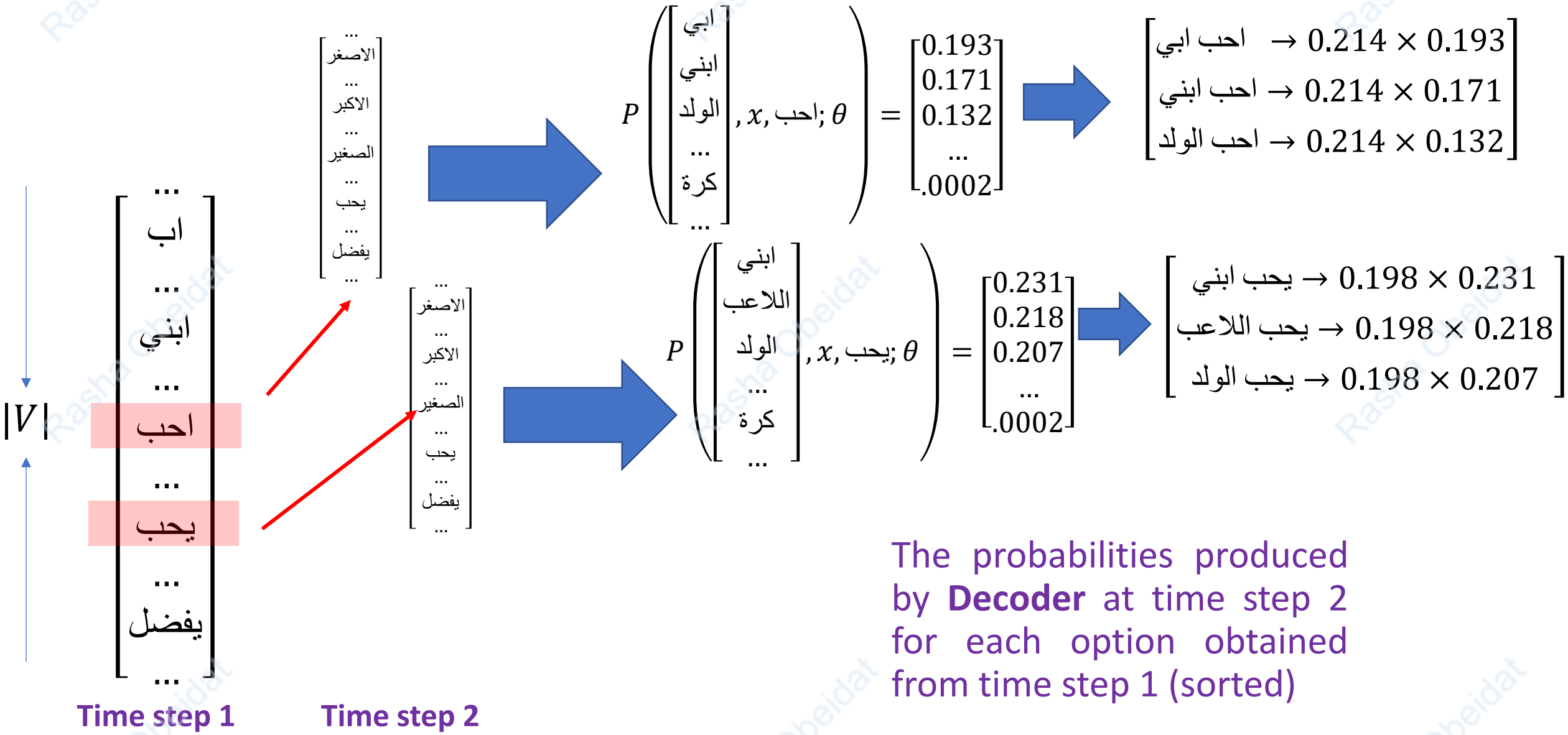
The probabilities produced by **Decoder** at time step 2 for each option obtained from time step 1 (sorted)

My youngest son likes to play football <s> → <s> يحب ابني الصغير لعب كرة القدم



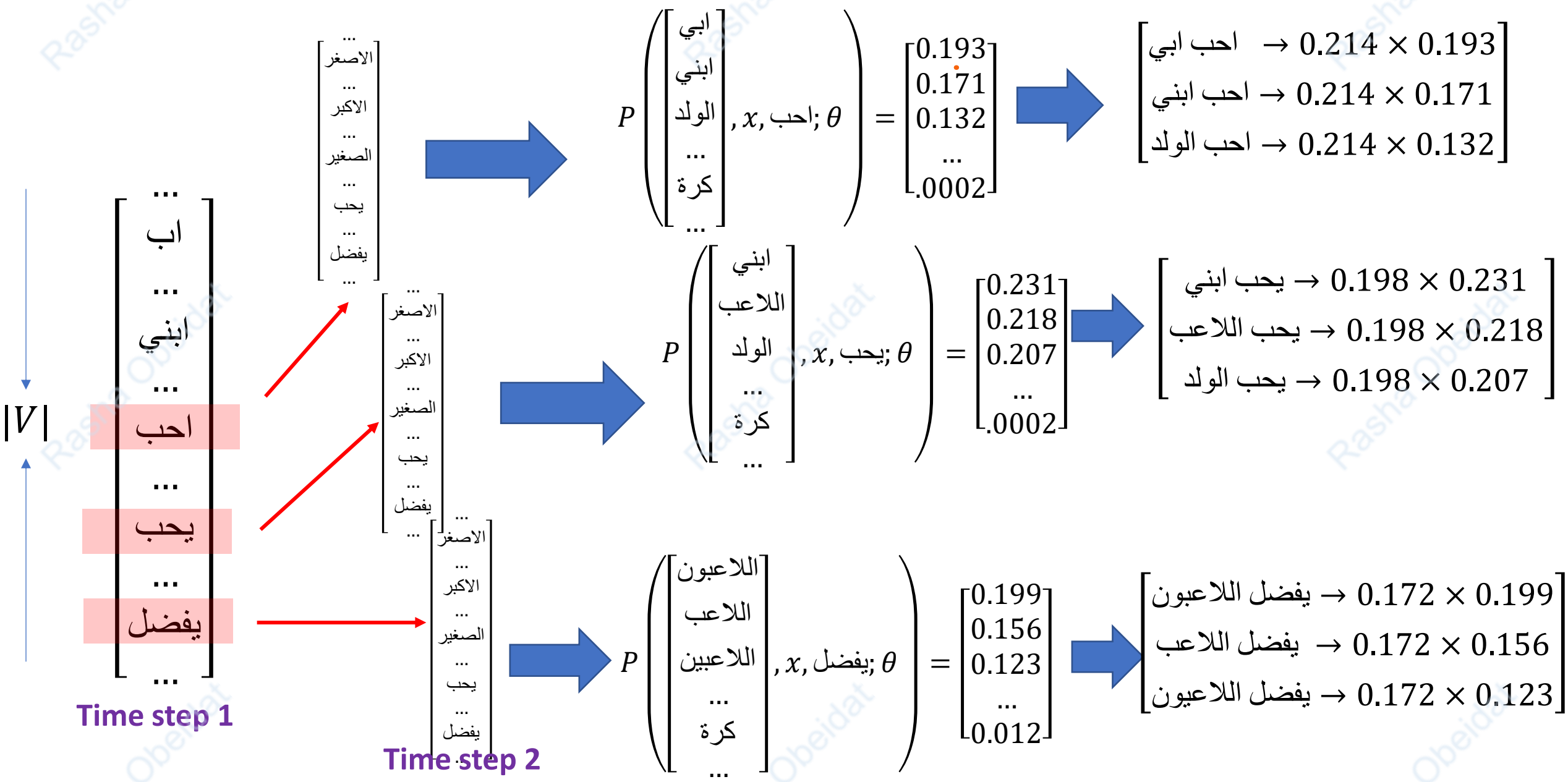
The probabilities produced by **Decoder** at time step 2 for each option obtained from time step 1 (sorted)

My youngest son likes to play football <s> → <s> يحب ابني الصغير لعب كرة القدم

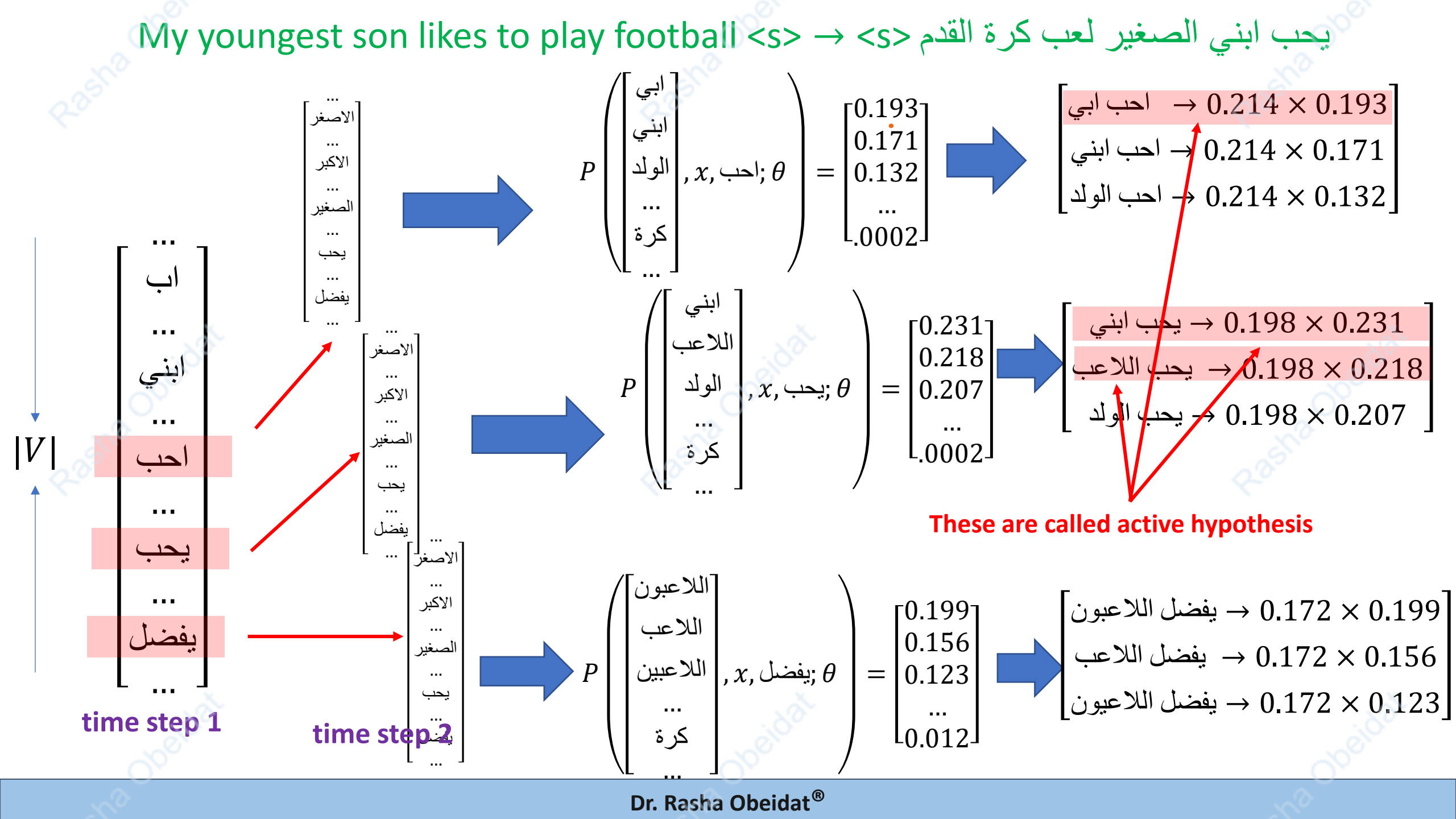


The probabilities produced by **Decoder** at time step 2 for each option obtained from time step 1 (sorted)

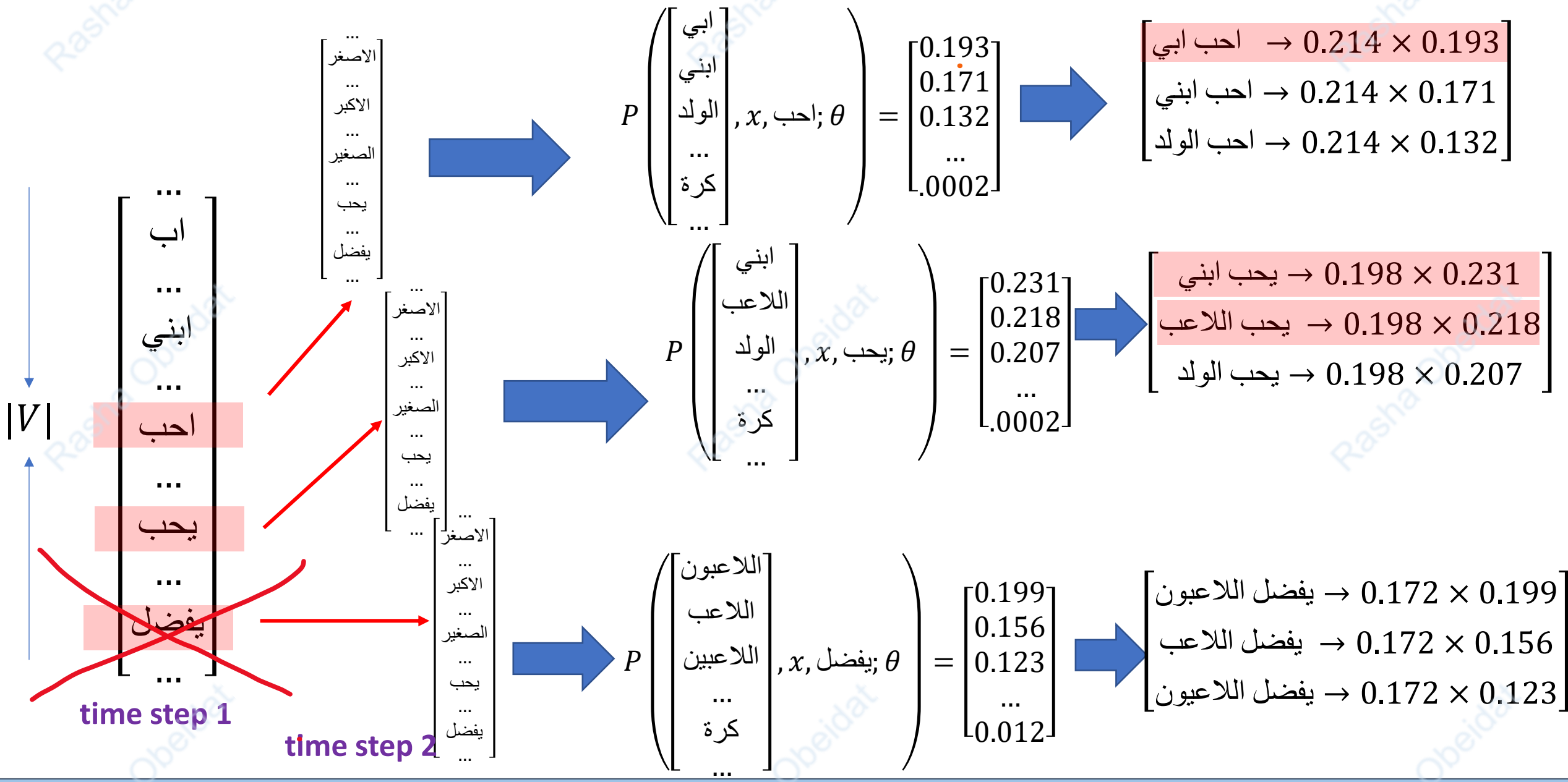
My youngest son likes to play football <s> → <s> يحب ابني الصغير لعب كرة القدم



My youngest son likes to play football <s> → <s> يحب ابني الصغير لعب كرة القدم



My youngest son likes to play football <s> → <s> يحب ابني الصغير لعب كرة القدم



Beam search

- What happens if $b = 1$? If $b = |V|$?

Rasha Obeidat

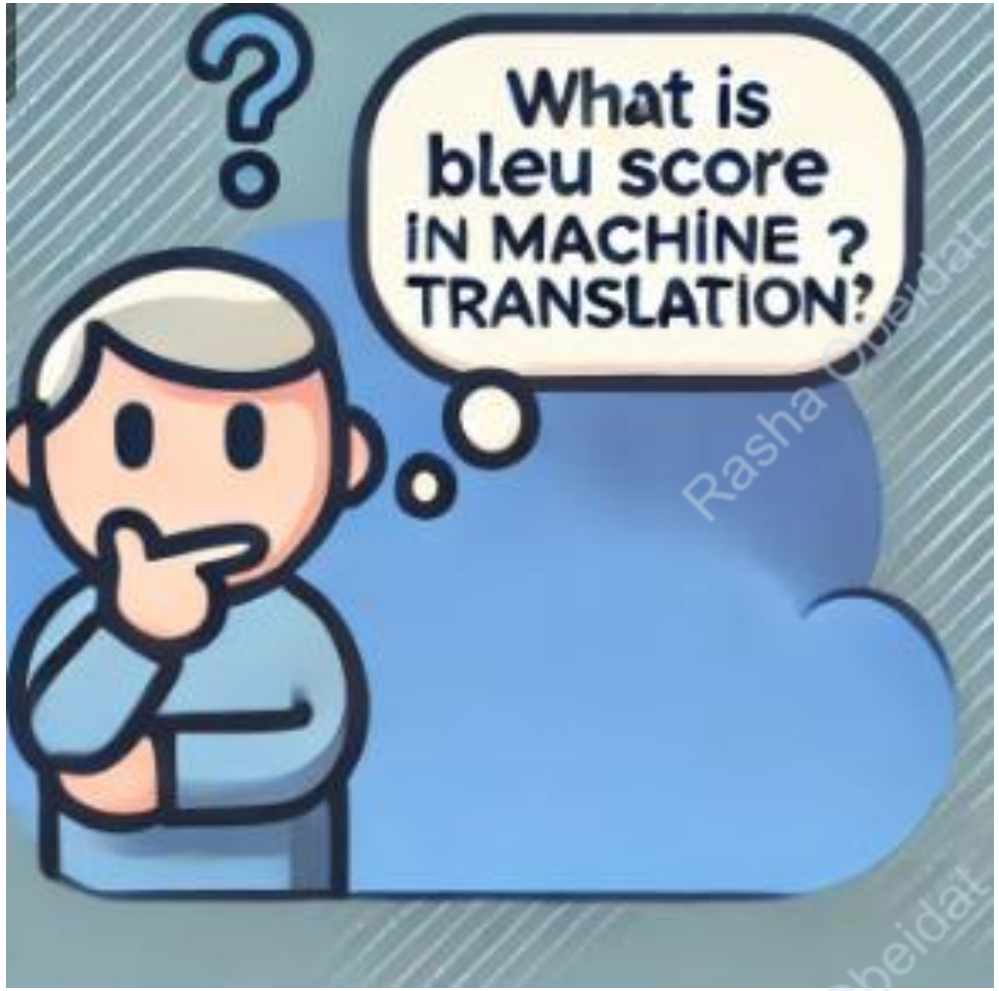
Overview of MT Evaluation Metrics

- Why Evaluate Machine Translation?
  - Assess the **quality** of machine-generated translations compared to reference translations.
  - Ensure translations are **accurate**, **fluent**, and **adequate** for the intended purpose.
- **Types of Metrics:**
  - **Automatic Metrics:**
    - Fast and scalable, rely on mathematical comparison with reference translations.
    - Examples: **BLEU, ROUGE, COMET**.
  - **Human Evaluation:**
    - Gold standard for assessing translation quality.
    - Measures fluency, adequacy, and naturalness.
- It is good to use more than one evolution metric to capture different aspects of the generated translation

MT Evaluation Metrics: BLEU

BLEU (Bilingual Evaluation Understudy)

- Measures ***n-gram precision***, which is n-gram overlap between the machine translations and reference translations.



MT Evaluation Metrics: BLEU

- **BLEU Formula**: Precision of n-grams weighted with a brevity penalty.

$$\boldsymbol{BLEU = BP \times exp(\sum_{n=1}^{N} w_n \times log(p_n))}$$

$$\boldsymbol{= BP \times exp(\frac{1}{N} \sum_{n=1}^{N} log(p_n))}$$

When using uniform n-gram wights

  - w_n: Weight for each n-gram size.
  - BP: Brevity penalty.
  - N : **maximum n length** that is considered for calculating the score (in practice N=4 is used).
  - p_n: Precision for n-grams .
  - BLEU typically uses the ***natural logarithm (ln)*** as in the original paper.

$$\boldsymbol{p_n = \frac{Number\ of\ overlapping\ n-gram}{Total\ number\ of\ n-gram\ in\ MT\ output}}$$

MT Evaluation Metrics: BLEU

- **How w_n is computed?**
- By default , the weights of all N-grams are equal (assuming Uniform weights):

$$w_n = \frac{1}{\mathrm{N}}$$

- For example: if $N = 4$ then w_1, w_2, w_3 and w_4 are 0.25
- **Why *Uniform* Weights?**
  1. Ensure **simplicity**: Each n-gram level is treated equally, making the computation straightforward.
  2. Equal weighting balances the trade-off between higher n-grams that capture more context but are rarer, and lower n-grams that are more frequent but provide less context.
- **Can custom Weights used?**
  1. Giving higher weights to Longer n-grams (e.g., $[w_1, w_2, w_3, w_4] = [0.1, 0.2, 0.3, 0.4]$): helps capturing fluency and context since they reflect phrase-level and syntactic coherence.
  2. Giving higher weights to Shorter n-grams (e.g., $[w_1, w_2, w_3, w_4] = [0.3, 0.3, 0.2, 0.2]$): is better suited for lexical accuracy as they evaluate whether individual words match the reference.
  3. Rarely used in practice!

MT Evaluation Metrics: BLEU

- **How BP is computed?**
  - The **Brevity Penalty (BP)** in BLEU is designed to penalize overly short translations.

$$BP = \begin{cases} 1 & if\ c > r \\ e^{\left(1-\frac{r}{c}\right)} & if\ c \leq r \end{cases}$$

  - c: Length of the machine translation (hypothesis).
  - r: Length of the reference translation.

  - When hypothesis translation (generated) is **longer** than or equal to the reference, BP=1 (no penalty is applied).
  - When the hypothesis translation is shorter than the reference, BP is less than 1.
- **Why Use BP?**
  - Without BP, BLEU might favor shorter translations that only include the most probable words but lack grammatical correctness or full meaning.
  - BP encourages the model (when used for model selection) that produce translations that are closer in length to the reference.

MT Evaluation Metrics: BLEU

- **Example:**
  Reference: "The cat is on the mat."
  MT Output: "The mat has a cat."
- Assume $N = 4$
- **Step 1**: Extract n-grams
  - Unigrams (1-grams):
    Reference: ["The", "cat", "is", "on", "the", "mat"]
    MT Output: ["The", "mat", "has", "a", "cat"]

$$p_1 = \frac{3}{5} = 0.6$$

  - Bigrams (2-grams):
    Reference: ["The cat", "cat is", "is on", "on the", "the mat"]
    MT Output: ["The mat", "mat has", "has a", "a cat"]

$$p_1 = \frac{1}{4} = 0.25$$

  - Trigrams (3-grams) and 4-grams:
    - Definitely p_3 and p_4 are zeros

MT Evaluation Metrics: BLEU

- **Step 2:** compute BP
  - Here, $c = 5$ and $r = 6$, so $c \leq r$:
  - $BP = e^{\left(1-\frac{r}{c}\right)} = e^{1-\frac{5}{6}} = e^{-0.2} \approx 0.81$

$$\boldsymbol{BLEU = BP \cdot exp(\sum_{n=1}^{N} w_n \, log(p_n))}$$

- Step 3. Compute BLEU

$$BLEU = 0.81 \times exp(\frac{1}{4} \times (\log 0.6 + \log(0.25) + \log(0) + \log(0)))$$

- Since $\log(0)$ is not defined , we use $\log(10^{-12})$as a smoothing term

$$BLEU = 0.81 \times exp(\frac{1}{4} \times (\log 0.6 + \log(0.25) + \log(10^{-12}) + \log(10^{-12}))) = 5.12 \times 10^{-7}$$

MT Evaluation Metrics: BLEU

- **Example1:**

  Reference: "The cat is on the mat."

  MT Output: "The mat has a cat.“

  BlEU-4= 0.00011 *(which is two low even though it is a completely correct translation)*

- **Example2:**

  Reference: "The cat is on the mat."

  MT Output: " The cat is on the market. "

  BlEU-4=0.556 *which is high even though the translation has a critical issue (which is where the cat actually is*)

  - **Solution:** To capture such semantic errors, consider using additional evaluation metrics simultaneously, such as **ROUGE** and **COMET**.

MT Evaluation Metrics: BLEU

- **Strengths**:
  - Efficient and scalable for large datasets.
  - Works well when reference translations are lexically similar to the machine output.
- **Weaknesses**:
  - Penalizes creative or valid translations that don't closely match the reference (e.g., synonyms, paraphrases).
  - Ignores semantic meaning and grammatical correctness.
  - Not sensitive to word order beyond the n-gram size.

Other Tasks Where BLEU Can Be Used

- **Text Summarization:** Evaluates overlap with reference summaries (better for extractive tasks).
- **Paraphrase Generation:** Compares generated paraphrases to reference ones.
- **Caption Generation:** Assesses the quality of generated captions for images or videos.
- **Code Generation:** Evaluates code generated from natural language descriptions.
- **Text Simplification:** Checks overlap with reference simplified texts.
- **Chatbot Response Generation:** Evaluates chatbot response relevance, though limited for context.
- **Note:** Similar to using BLEU to evaluate MT models, for these tasks BLEU focuses on n-gram overlap and may miss semantic nuances—consider complementing it with metrics like **ROUGE** or **COMET**.

Overview of MT Evaluation Metrics

- **ROUGE (Recall-Oriented Understudy for Gisting Evaluation)**
  - Compares recall of overlapping words, phrases, or n-grams between MT and references.
  - Variants: ROUGE-1 (unigrams), ROUGE-L (longest common subsequence).
  - Strengths: Captures coverage of reference.
  - Weaknesses: Focuses on recall, less sensitive to precision.

Rasha Obeidat

Overview of MT Evaluation Metrics

- **COMET (Cross-lingual Optimized Metric for Evaluation of Translation)**
  - A neural metric that uses multilingual embeddings and pretrained models.
  - Incorporates semantic context and reference alignment.
  - Strengths: More accurate, aligns well with human judgment.
  - Weaknesses: Computationally expensive, requires training.

Rasha Obeidat

- These slides are the intellectual property of **Dr. Rasha Obeidat®**. If you wish to use them for teaching **your course**, please contact the author for permission.
  The slides may **not be modified or redistributed** without explicit permission.
- **Contact:**
  - Obeidat.rm@gmail.com
  - Rmobeidat@just.edu.jo



Rasha Obeidat